System Domain - HanziApp

The HanziApp System Domain defines three things: the system context (what is inside, what is outside, and which external systems it talks to), the characterization of its actors, and the business rules that govern learning.

It builds on the requirements and user stories of Deliverable 1 (September 30, 2026), and adds the design decisions made after that deliverable:

- A deck can belong to several lists.
- A user can hold several roles; teachers also study.
- A flashcard's meaning is revealed step by step.
- Senses and Chinese example sentences are shared across languages; only their translations change.
- Each course appears to its students as a read-only list.

Identifiers keep their original prefixes so they match the requirements document: RF (functional requirement), HU (user story), RN (business rule) and RNF (non-functional requirement). RN-01 to RN-16 keep their numbers, new rules continue from RN-17, and a rule split into parts uses decimals (RN-15.1, RN-15.2).

1. System context

HanziApp is a responsive web application that helps Spanish and English speakers memorize the vocabulary and handwriting of HSK 3.0, levels 1 to 6. It interacts with four human actors and a time-based scheduler, and depends on six external systems. The advanced band of HSK 3.0 (levels 7 to 9) is out of scope.

1.1 Purpose

The system solves three problems for a Chinese learner:

- **What to study:** it offers the official HSK 3.0 vocabulary, organized into lists and decks.
- **When to review:** it schedules every word with FSRS and every character with an in-house motor-memory algorithm.
- **How to keep the habit:** it tracks the daily streak, learned words and characters, and progress per level.

It also gives teachers a way to assign vocabulary to their courses and follow the group's progress.

1.2 Domain glossary

These terms are used throughout the System Domain and the other deliverables.

Term	Meaning in HanziApp
HSK 3.0	The new Chinese Proficiency Test, based on the official syllabus released in 2025 and applied from 2026. It has 9 levels in three bands (elementary 1–3, intermediate 4–6, advanced 7–9); HanziApp covers levels 1 to 6.
Hanzi	A Chinese character, such as 好.
Word	A vocabulary item made of one or more hanzi with a specific reading, such as 你好 (nǐ hǎo).
Pinyin	The official romanization of Mandarin. It marks tones with accents (hǎo) or with numbers (hao3).
Tone	The pitch pattern of a syllable. Mandarin has four tones and a neutral tone, and a different tone can change the meaning.
Sense	One meaning of a word, with its part of speech. 还 (hái) has several senses, such as "still" and "also".
Stroke order	The standard order and direction in which the strokes of a character are written.
Deck	A group of words studied together, such as "Greetings".
List	A group of decks. System lists are HSK 1 to HSK 6; users can also create their own.
Active recall	Retrieving an answer from memory before seeing it, which is how a flashcard works.

Spaced repetition	A study technique that schedules each review just before the learner is likely to forget the item.
FSRS	Free Spaced Repetition Scheduler: the algorithm that computes each word's next review date.
Stability	In FSRS, the number of days until the probability of remembering a word falls to 90%.
Due review	A word or character whose next review date has arrived.
Motor memory	The memory of how to write a character by hand, trained in the writing practice.
Streak	The number of consecutive days on which the user completed at least one study session.

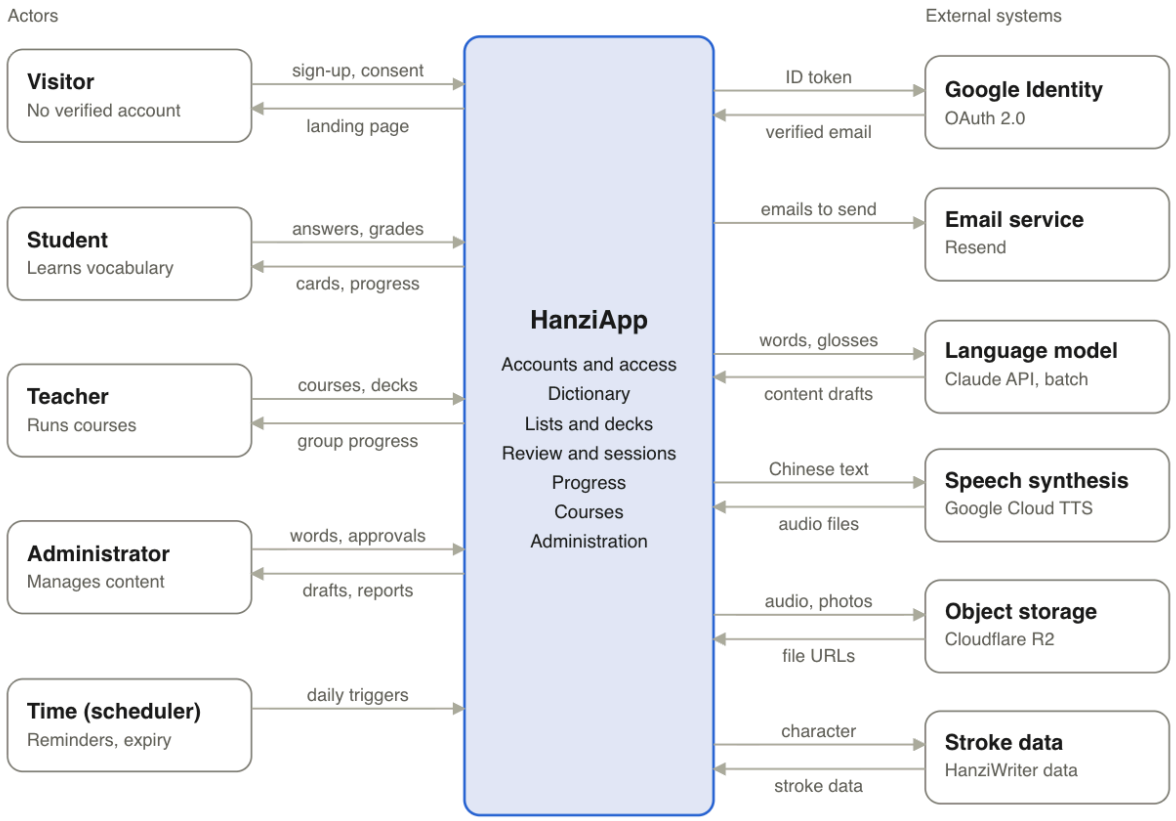
1.3 System boundary

Seven subsystems sit inside the system:

1. Accounts and access
2. Content and dictionary
3. Lists and decks
4. Review and study sessions (flashcards and writing)
5. Progress and motivation
6. Courses
7. Content and user administration

Google identity, email delivery, AI content generation, speech synthesis, file storage and stroke data are handled by external systems.

HanziApp exchanges data with five actors and six external systems



Each arrow is labeled with the data it carries.

Each arrow is labeled with the data it carries: arrows pointing into HanziApp are inputs and arrows pointing out are outputs.

1.4 External systems

External system	What it exchanges with HanziApp	Direction	Status	RF
Google Identity (OAuth 2.0)	The user's verified identity: email and name	Inbound	In use	RF-07, RF-08, RF-09
Transactional email service (Resend)	Verification, password reset, confirmation, request decision and reminder emails	Outbound	In use	RF-03, RF-05, RF-11, RF-22, RF-82, RF-87

Language model (Claude API, batch)	Receives words with their official gloss; returns senses, examples and translations as drafts	Both	In use	RF-102
Speech synthesis (Google Cloud TTS)	Receives Chinese text; returns audio for words and examples	Both	Planned	RF-37, RF-38
Object storage (Cloudflare R2)	Stores and serves audio files and profile photos	Both	Planned	RF-16, RF-37, RF-38
Character stroke data (HanziWriter)	Shape and order of each character's strokes	Inbound	Planned	RF-39, RF-61 to RF-65

The official HSK 3.0 vocabulary is not a connected system: it enters as a file through bulk import (RF-100).

The language model sits outside the boundary even though the generation pipeline currently runs as scripts. RF-102 moves the trigger into the administration module, so HanziApp calls the model the same way it calls any other external system.

1.5 Scope

In scope: the Must, Should and Could functional requirements of Deliverable 1. The first release of the semester commits to the Must requirements.

Out of scope:

- The HSK 3.0 advanced band, levels 7 to 9.
- Pronunciation recording and assessment (RF-85 and RF-86, Won't). They are documented as vision.
- Mock exams or HSK-style quizzes.

2. Actor characterization

HanziApp has four human actors and seven secondary actors. Human roles are cumulative: every verified account is a student account, and the teacher and administrator roles are added on top of it.

2.1 Human actors at a glance

Actor	Type	Who it is	How the role is obtained
Visitor	Primary	A person without an account, or with an account pending verification	Not a stored role: it is the absence of a verified account
Student	Primary	A user with a verified email who is learning Chinese	By verifying the email, or by signing up with Google
Teacher	Primary	A Chinese teacher (Confucius Institute, university programs) who can also study	An administrator approves their request (RF-87, RF-114)
Administrator / content manager	Primary	A HanziApp team member responsible for content and accounts	Another administrator assigns it (RF-112)

2.2 Actor generalization

Teacher and Administrator specialize Student: they inherit every student interaction and add their own. A visitor becomes a student once the email is verified.

[embed: node/6d589f32-d84b]

2.3 Actor profiles

The profile describes each human actor along the same dimensions. Values are the team's expected profile and will be validated with the onboarding questionnaire (RF-27 to RF-30).

Dimension	Visitor	Student	Teacher	Administrator

Technical skill	Basic	Basic; not a technical user (RNF-37)	Basic to intermediate	High; a member of the project team
Chinese level	Any, often a beginner	HSK 1 to 6; the first release has HSK 1 content	Advanced or native	Enough to check content against the official glossary
Typical use	One visit to evaluate the app and sign up	Short, frequent sessions, ideally every day	Around their classes, to assign decks and check progress	In batches, when loading or reviewing content
Main device	Phone (RNF-38)	Phone or computer; writes with a finger or a mouse (RNF-38, RNF-39)	Computer	Computer
Language	Spanish or English	Spanish or English, as chosen at sign-up	Spanish or English	Both, since they review content in both languages
Motivation	Find a tool to prepare for the HSK	Pass an HSK exam, study or travel in China, personal interest	Keep students practicing between classes	Publish correct content
Expected number	Every newcomer	The largest group; version 1 had 16 users from the Confucius Institute	A few per institution	The 5-person project team
Release 1 priority	Must	Must	Could	Must for content management

2.4 Visitor

- **Goals:** learn about the value proposition, create an account and verify the email.
- **Interactions:** landing page and language selector (RF-24, RF-25); sign-up with email or Google (RF-01, RF-02, RF-07, RF-14); email verification and resend (RF-04, RF-05); sign-in and password reset (RF-08, RF-10 to RF-12).
- **Constraints:** cannot use any student feature (RN-16.1). Must accept the data policy before the account is created (RN-17.1). A minor needs authorization from a legal guardian (RNF-33).

2.5 Student

- **Goals:** memorize the vocabulary and handwriting of their HSK level, keep a daily habit and know how ready they are for the exam.
- **Interactions:**
    - Profile and onboarding questionnaire (RF-15 to RF-23, RF-27 to RF-30).
    - Dictionary (RF-31 to RF-42).
    - Lists and decks (RF-43 to RF-53).
    - Flashcards, writing and sessions (RF-54 to RF-74).
    - Progress and motivation (RF-75 to RF-84).
    - Courses: enroll, leave and request the teacher role (RF-87, RF-90, RF-95).
- **Constraints:** cannot modify system or course lists and decks (RN-13). Only sees content approved in their study language (RN-14, RN-15.3). A suspended account cannot sign in (RN-22.1). Must finish a first session in under 3 minutes without help (RNF-37).

2.6 Teacher

- **Goals:** assign the vocabulary of their classes to their course and spot the students who need support.
- **Interactions:** course creation and invitation code regeneration (RF-88, RF-89); course decks and their assignment (RF-91, RF-92); individual and group progress (RF-93, RF-94). Keeps every student interaction.
- **Constraints:** only sees students enrolled in their courses (RN-47.1), and never their email or questionnaire answers (RN-47.2). If the role is removed, their courses become inactive (HU-112, RN-55).

2.7 Administrator / content manager

- **Goals:** publish correct content in both languages, maintain the system HSK lists and support users.
- **Interactions:**
    - Vocabulary: create, edit, deactivate and import (RF-96 to RF-101).
    - Content: generate with AI, validate, edit, approve and reject drafts (RF-99, RF-102 to RF-106).
    - User error reports (RF-107).
    - System decks (RF-108 to RF-110).
    - Users: search, roles, suspension and teacher requests (RF-111 to RF-115).

    - Usage metrics (RF-116).
- **Constraints:** their actions are recorded in the audit log (RN-49). Cannot remove the last administrator role nor suspend their own account (RN-18.3, RN-22.3). Cannot see passwords or questionnaire answers (HU-111), and metrics never identify an individual (HU-116).

2.8 Secondary actors

Actor	Type	Role in the system
Google Identity	External system	Authenticates the user and provides their verified email
Transactional email service	External system	Delivers emails in the user's study language
Language model	External system	Generates content drafts; never publishes (RN-14)
Speech synthesis	External system	Generates audio once (RNF-58)
Object storage	External system	Stores audio files and profile photos
Stroke data	External system	Provides the shape and order of each character's strokes
Time (scheduler)	Time actor	Sends reminders at 8:00 p.m. local time (RF-82), closes the day for the streak (RN-11), expires verification and reset links, and triggers the daily backup (RNF-10)

3. Business rules

HanziApp is governed by 96 business rules in eight categories, and each rule states exactly one thing. A rule from an earlier version that stated several things is split into numbered

parts (RN-15.1, RN-15.2…), so references to the original number stay valid. Three rules were added in this version to close gaps in data protection and courses (RN-53 to RN-55). RN-20, RN-21 and RN-35 were retired because they are security or interface requirements, not business rules (section 4.1).

Each rule is classified with the usual business-rule taxonomy:

Type	Meaning
Fact	A statement about the domain's structure or identity that is always true
Constraint	Something that must or must not happen; the system rejects anything that breaks it
Action enabler	A condition that automatically triggers an action
Inference	A conclusion the system derives from other facts
Computation	How a value is calculated

3.1 Accounts and access

ID	Rule	Type	Change	Source
RN-16.1	An account created with email and password stays pending, and cannot use any student feature until the email is verified.	Constraint	Deliverable 1	RF-03 to RF-06
RN-16.2	An email verification link expires 24 hours after it is sent.	Constraint	Deliverable 1	RF-04

RN-16.3	An account created with Google is considered verified from the moment it is created.	Inference	Deliverable 1	RF-07
RN-17.1	No account is created, with email or Google, without explicit acceptance of the current data policy.	Constraint	New	RF-02, RNF-26
RN-17.2	Accepting the data policy records the date, the time and the version accepted.	Action enabler	New	RNF-26
RN-18.1	Roles are cumulative: every verified account has the student role, and the teacher and administrator roles are added to it.	Fact	New	Design decision
RN-18.2	Removing a role never deletes the user's progress.	Constraint	New	HU-112
RN-18.3	The administrator role cannot be removed from the last administrator.	Constraint	New	HU-112
RN-19.1	One email maps to exactly one account.	Constraint	New	HU-07
RN-19.2	Signing in with Google with an email that is already registered links both access methods to that account and marks it as verified.	Action enabler	New	HU-09
RN-22.1	A suspended account cannot sign in by any method.	Constraint	New	HU-113

RN-22.2	Suspending an account ends all its active sessions.	Action enabler	New	HU-113
RN-22.3	Nobody can suspend their own account.	Constraint	New	HU-113
RN-22.4	Reactivating a suspended account keeps all its progress.	Constraint	New	HU-113
RN-23.1	Deleting an account permanently removes its personal data, lists, decks, progress and enrollments.	Action enabler	New	HU-22
RN-23.2	Signing up again with the email of a deleted account creates a new account with no progress.	Inference	New	HU-22
RN-53	When a new version of the data policy is published, existing users must accept it at their next sign-in before they can continue.	Constraint	New	Ley 1581 de 2012, RNF-26
RN-54	Every user can consult and correct their personal data and request its deletion at any time; deletion follows RN-23.1.	Constraint	New	Ley 1581 de 2012 (art. 8), RF-15, RF-22

3.2 Content

ID	Rule	Type	Change	Source
RN-14	No AI-generated content is visible to users until an administrator has reviewed and approved it.	Constraint	Deliverable 1	RF-102 to RF-105

RN-15.1	Each user has one study language, Spanish or English, chosen at sign-up.	Fact	Deliverable 1	RF-14
RN-15.2	The study language sets the language of the interface, senses, translations, meaning search and emails.	Inference	Deliverable 1	RF-17
RN-15.3	A word is shown in a language only when its content is approved in that language.	Constraint	Deliverable 1	RF-17
RN-15.4	Changing the study language does not affect progress.	Constraint	Deliverable 1	RF-17
RN-24	The hanzi, the pinyin, the senses with their part of speech, and the Chinese example sentences are shared across languages; only the translations change per language.	Fact	New	Design decision
RN-25.1	Each word's content has one status per language: pending, draft, approved or rejected.	Fact	New	RF-99
RN-25.2	There is at most one approved content per word and language.	Constraint	New	HU-105
RN-25.3	A draft with validation failures cannot be approved.	Constraint	New	HU-105, HU-106
RN-26.1	A word is identified by its hanzi and its pinyin, so the same hanzi with different readings, such as 还 (hái and huán), are different words.	Fact	New	HU-31, HU-96

RN-26.2	Each word belongs to exactly one HSK 3.0 level, from 1 to 6.	Constraint	New	HU-96
RN-27.1	A deactivated word does not appear in the dictionary, decks or sessions, and does not count in any metric.	Constraint	New	HU-98
RN-27.2	Deactivating a word keeps every user's progress on it, and that progress counts again if the word is reactivated.	Constraint	New	HU-98
RN-28	Each word is made of one or more characters in a fixed order, and one character can be part of many words.	Fact	New	RF-40, RF-65

3.3 Lists and decks

ID	Rule	Type	Change	Source
RN-03	Decks and lists only group words: deleting a deck or a list, or removing a word from a deck, never deletes the user's progress on that word or its characters.	Constraint	Deliverable 1	RF-46, RF-50, RF-51
RN-13	System lists (HSK 1 to 6) are read-only for students and teachers; only administrators can modify them.	Constraint	Deliverable 1	RF-43, RF-108 to RF-110
RN-29.1	A personal deck can belong to several of its owner's lists, with a separate position in each list.	Fact	New	Design decision

RN-29.2	Deleting a list also deletes the decks that belong to no other list.	Action enabler	New	Design decision, HU-46
RN-29.3	Deleting a deck removes it from every list it belongs to.	Action enabler	New	HU-50
RN-30.1	A list name is unique among its owner's lists.	Constraint	New	HU-44
RN-30.2	A deck name is unique within each list and within each course.	Constraint	New	HU-47, HU-91
RN-31.1	A system list contains only words of its own HSK level.	Constraint	New	HU-108, HU-109
RN-31.2	Each word of a system list is in exactly one of its decks.	Constraint	New	HU-109
RN-31.3	An empty system deck is not shown to users.	Constraint	New	HU-31
RN-32	Copying a system deck creates an independent personal deck: later changes to the original do not affect the copy.	Constraint	New	HU-52
RN-33.1	Each course appears to its enrolled students as a read-only list containing the assigned decks.	Inference	New	Design decision, HU-92
RN-33.2	The teacher's changes to an assigned deck are visible to students immediately.	Inference	New	HU-92

RN-33.3	Leaving a course removes the course list from the student's lists.	Action enabler	New	HU-95
RN-34	A study session cannot be started on an empty deck.	Constraint	New	HU-47

3.4 Review and writing

ID	Rule	Type	Change	Source
RN-01	Each user has a single FSRS review state per word (stability, difficulty and next review date), independent of the deck where the word is reviewed.	Constraint	Deliverable 1	RF-57
RN-02	Each user has a single writing review state per character, independent of the word where it is practiced and of the recognition state.	Constraint	Deliverable 1	RF-66
RN-05.1	Grading a card Easy (remembered) records an FSRS success, and its interval grows.	Computation	Deliverable 1	RF-56
RN-05.2	Grading a card Hard (not remembered) records an FSRS lapse, and its interval resets.	Computation	Deliverable 1	RF-56
RN-36.1	A card graded Hard comes back after at least 3 other cards, or immediately if it is the only one left.	Action enabler	New	HU-58

RN-36.2	After three Hard grades in the same session, a card leaves the session and stays due for the next day.	Action enabler	New	HU-58
RN-37.1	The new-word limit counts the new words studied per day, in the user's time zone, across all their decks.	Computation	New	RF-59
RN-37.2	The new-word limit defaults to 10 and accepts values from 0 to 50.	Constraint	New	RF-21
RN-37.3	Words seen in free review do not count toward the new-word limit.	Constraint	New	RF-59, RF-69
RN-38.1	The next writing review is computed from the errors, the hints used and the writing mode.	Computation	New	RF-66
RN-38.2	With the same performance, the next interval is shorter in easy mode than in hard mode.	Computation	New	HU-62, HU-66
RN-38.3	Marking "I don't know" resets the character's writing interval.	Action enabler	New	HU-66
RN-39	In hard mode, three consecutive errors on the same stroke show a hint for that stroke, and the hint counts in the next-review calculation.	Action enabler	New	RF-64

3.5 Study sessions

ID	Rule	Type	Change	Source

RN-04.1	A scheduled session's queue is built when it starts: first the deck's words due today or earlier, then new words up to the daily limit.	Computation	Modified	RF-68
RN-04.2	A session stores its queue so it can be resumed.	Constraint	Modified	RF-72
RN-04.3	Reviewing a word in any deck removes it from the due reviews of every deck.	Inference	Modified	RF-68
RN-04.4	A word already studied is never presented as new in another deck.	Constraint	Modified	RF-68
RN-12.1	An abandoned session stays in progress with its position: the current card and, in writing, the current character.	Constraint	Deliverable 1	RF-72
RN-12.2	There is at most one in-progress session per deck and practice type.	Constraint	Deliverable 1	RF-73
RN-12.3	A scheduled session is recalculated when it is resumed, while a free review keeps its order until it is finished or discarded.	Action enabler	Deliverable 1	RF-72, RF-73
RN-40	Free review presents every word of the deck in random order and does not change any review state.	Constraint	New	RF-69
RN-41	The all-due-reviews session gathers the due reviews from all the user's decks, each word once and with no new words.	Computation	New	RF-70

RN-42.1	Each grade and each completed character is saved on the server immediately.	Action enabler	New	RF-71
RN-42.2	Discarding a session does not undo the grades already recorded.	Constraint	New	HU-73
RN-43	Deleting a deck or a list, or leaving a course, discards the in-progress sessions of the affected decks.	Action enabler	New	HU-46, HU-50, HU-95

3.6 Progress and motivation

ID	Rule	Type	Change	Source
RN-06	A word is learned when its FSRS stability is 21 days or more.	Inference	Deliverable 1	RF-77
RN-07	A character is learned when its writing interval is 21 days or more and it has been practiced in hard mode.	Inference	Deliverable 1	RF-78
RN-08	Progress in an HSK level is the percentage of learned words over the active words of that level.	Computation	Deliverable 1	RF-79
RN-09	Metrics count distinct words and characters, without duplicates, even when they appear in several decks.	Computation	Deliverable 1	RF-77 to RF-80
RN-10.1	An active day is a calendar day, in the user's time zone, on which they complete at least one session of any scope, even with the same deck as on previous days.	Inference	Modified	RF-75

RN-10.2	A session counts for the day it is completed, and an abandoned session does not count.	Constraint	Modified	RF-75
RN-10.3	The active day is stored when the session completes and is not recalculated if the user changes time zone.	Action enabler	Modified	HU-19
RN-11	The streak is the number of consecutive active days, and it resets to zero when a day passes with no activity.	Computation	Deliverable 1	RF-75
RN-44	The longest streak is the highest value the streak has reached, and it never decreases.	Computation	New	RF-76
RN-45	Each badge is awarded at most once per user, even if the milestone is reached again.	Constraint	New	HU-84

3.7 Courses

ID	Rule	Type	Change	Source
RN-46.1	Enrolling requires a valid invitation code and agreeing to share progress with the teacher.	Constraint	New	HU-90
RN-46.2	Regenerating a course's code invalidates the previous one without affecting students already enrolled.	Action enabler	New	HU-89
RN-47.1	A teacher only sees the progress of students enrolled in their courses: streak, active days,	Constraint	New	RNF-32, HU-93

	learned words and progress on assigned decks.			
RN-47.2	A teacher never sees a student's email or questionnaire answers.	Constraint	New	HU-93
RN-47.3	A teacher stops seeing a student's progress once the student leaves the course.	Constraint	New	HU-95
RN-48.1	A user can have at most one pending teacher-role request.	Constraint	New	HU-87
RN-48.2	A teacher-role request must state the institution.	Constraint	New	HU-87
RN-55	An inactive course accepts no new enrollments and cannot be edited; its enrolled students keep the course list and can still study its decks.	Constraint	New	HU-112

3.8 Administration

ID	Rule	Type	Change	Source
RN-49	The audit log records, with author and date, every creation, edit and deactivation of a word; every approval and rejection of content; every system deck created; every role change and suspension; and every decision on a teacher request.	Action enabler	New	RNF-24
RN-50	Rejecting content, suspending an account and rejecting a teacher request each require a reason.	Constraint	New	HU-106, HU-113, HU-115

RN-51.1	AI generation works in batches of up to 100 words in one language.	Constraint	New	HU-102
RN-51.2	A generation batch skips the words that already have approved content in its language.	Constraint	New	HU-102
RN-52.1	An import never creates a duplicate word, that is, one with the same hanzi and pinyin.	Constraint	New	HU-100
RN-52.2	An import keeps the valid rows even when other rows fail.	Constraint	New	HU-101

4. Changes from Deliverable 1

The decisions made after Deliverable 1 require adjusting five user stories, creating two new ones and modifying two rules. Each new RF and HU contains a single action, as the professor requested.

RF / HU	Required change	Rule
RF-55 / HU-55	Rewrite the reveal as a step-by-step flow: tap for pinyin, tap for meaning, and a Sentences button for the examples. The grading buttons appear once the meaning is visible, and the space bar advances one step.	Formerly RN-35
New RF / HU	Add one of the user's own decks to another of their lists.	RN-29.1
New RF / HU	Remove a deck from a list without deleting it, as long as it remains in another list.	RN-29.1

HU-46	Deleting a list deletes only the decks that are not in another list. The confirmation states how many decks will be deleted and how many will be kept.	RN-29.2
HU-50	Clarify that deleting a deck removes it from every list where it appears.	RN-29.3
RF-112 / HU-112	Split into "Role assignment" and "Role removal". Criterion 3 becomes "when the teacher role is removed".	RN-18.1, RN-18.3
HU-92	Specify that the assigned deck appears in the course list each enrolled student sees.	RN-33.1
RN-04	The session stores its queue so it can be resumed, instead of never storing it.	RN-04.2
RN-10	The active day is stored when the session completes and is not recalculated when the time zone changes.	RN-10.3

4.1 Rules retired or moved

Four statements left the business rules because they describe security policy, interface behavior or technical limits. Their numbers are not reused.

Former rule	Content	Moves to	Reason
RN-20	Five failed sign-in attempts in 15 minutes lock the account for 15 minutes	RNF-22	Security policy
RN-21	A reset link is single-use, expires after one hour, and using it signs out other devices	RNF-23	Security policy

RN-35	Step-by-step reveal of a flashcard	RF-55, rewritten	Interface behavior
Part of RN-52	Import files must be CSV or JSON of up to 10 MB	A new RNF on import limits	Technical limit

4.2 Rules added in this version

Rule	Why it was added
RN-53	Ley 1581 de 2012 requires informed consent; if the policy changes, earlier consent no longer covers the new terms.
RN-54	Ley 1581 de 2012 (art. 8) gives data subjects the right to know, update and correct their data, to access it free of charge and to request its deletion.
RN-55	HU-112 makes a course inactive when its teacher loses the role, but no rule said what students see afterwards.

5. References

- Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587. https://cancilleria.gov.co/sites/default/files/Normograma/docs/ley_1581_2012.htm
- HanziApp team. (2026). *Deliverable 1: Requirements and user stories*. Ingeniería de Software II, Universidad Nacional de Colombia.
- Mandarin Bean. (2026, April 7). *New HSK vocabulary (2026)*. Retrieved October 9, 2026, from https://mandarinbean.com/new-hsk-vocabulary/
- Open Spaced Repetition. (n.d.). *The algorithm*. Awesome FSRS wiki, GitHub. Retrieved October 9, 2026, from https://github.com/open-spaced-repetition/awesome-fsrs/wiki/The-Algorithm
- Wiegers, K., & Beatty, J. (2013). *Software requirements* (3rd ed.). Microsoft Press.

The HSK 3.0 vocabulary loaded in HanziApp comes from the Mandarin Bean lists, which follow the 2025 official syllabus. Their HSK 1 list has 300 words, the same number loaded in the database, and their English glosses are the official meaning anchor for content

generation. The business-rule taxonomy in section 3 follows Wiegers and Beatty, and the definition of stability in the glossary follows the FSRS documentation.

CRC Cards

These are HanziApp's 22 main classes, one CRC card each. A class is considered main when it has behavior of its own and takes part in several user stories, or when it is the parent of classes that do. Classes that only store a record (PolicyAcceptance, AuditLog, ImportError, among others) appear as collaborators instead of having a card.

Every card follows the standard CRC format: a header with the class name, its type, its superclass and its subclasses; a one-line description; and a two-column body with the responsibilities on the left and the collaborators on the right. Responsibilities state what the class knows or does; collaborators are the classes it needs to ask for something in order to fulfill them.

There are three kinds of class. **Entity** classes hold domain information, such as User or Deck. **Abstract** classes hold what several entities share and are never created directly, such as ReviewState. **Control** classes encapsulate behavior, such as FSRSScheduler: this is how RNF-53 is met, since it requires the scheduling algorithms to be decoupled modules. The superclasses and subclasses match the class diagram.

Class names are in English. The traceability table at the end maps each one to its Spanish name in the domain model.

1. Accounts and access

User

Type: entity · **Superclass:** — · **Subclasses:** —

A person with an account: their identity, access methods, roles, settings and account status.

Responsibilities	Collaborators
Know its name, email and account status: pending, verified or suspended	—

Know its roles and answer whether it can perform an action	Role
Know its access methods and link Google when the email matches	AccessMethod
Record acceptance of the current data policy	PolicyAcceptance, DataPolicy
Ask for acceptance of a new data-policy version at the next sign-in, before anything else	DataPolicy, PolicyAcceptance
Let the user consult and correct their personal data	LearnerProfile
Know its settings: study language, time zone, daily goal, new-word limit and reminders	—
Convert an instant into its local day according to its time zone	—
Store its questionnaire answers and recommend the starting deck based on its declared level	LearnerProfile, DeckList
Request the teacher role	TeacherRequest
Suspend and reactivate itself, leaving a record	AuditLog
Delete itself along with its personal data, lists, decks, progress and enrollments	DeckList, Deck, RecognitionState, WritingState, Enrollment

2. Content

Word

Type: entity · **Superclass:** — · **Subclasses:** —

An HSK 3.0 vocabulary item with a specific reading, such as 你好 (nǐ hǎo); the center of all content.

Responsibilities	Collaborators
Know its hanzi, pinyin, HSK level, audio and whether it is active	—
Know its senses and examples, in order	Sense, Example
Know the characters it is made of, in order	Character, WordComposition
Provide its content in a language only if it is approved in that language	LanguageContent
Express its pinyin in the search forms: with tone marks, with numbers and without tones	—
Deactivate and reactivate itself without affecting users' progress	AuditLog
Receive error reports about its content	ErrorReport

TranslatableElement

Type: abstract · **Superclass:** — · **Subclasses:** Sense, Example

What senses and example sentences share: they are the same in every language and have one translation per language.

Responsibilities	Collaborators
Provide its translation in a given language	Translation
Ensure it has at most one translation per language content	Translation, LanguageContent
Keep its Chinese text and order the same in every language	—

Dictionary

Type: control · **Superclass:** — · **Subclasses:** —

The search service over the vocabulary; it holds no data of its own.

Responsibilities	Collaborators
Search by hanzi: exact match first, then words containing the character, ordered by level	Word
Search by pinyin with or without tones, with marks or numbers, ignoring spaces and case, and treating lü, lv and lu: as equivalent	Word
Search by meaning in the study language, ignoring accents and case, using the first sense to rank results	LanguageContent, User
Filter results by one or several HSK levels	Word
Exclude words that are deactivated or have no approved content in the user's language	Word, LanguageContent

List the words that contain a character	Character, WordComposition

LanguageContent

Type: entity · **Superclass:** — · **Subclasses:** —

A word's content in one language, Spanish or English, with its editorial status from draft to approved.

Responsibilities	Collaborators
Know its language, its status (pending, draft, approved or rejected) and its origin (AI, manual or imported)	—
Group the translations of the senses and examples in its language	Translation, TranslatableElement
Request its validation when created and every time it is edited	ContentValidator, ValidationResult
Get approved only if it has no failures, replacing the previously approved content in the same language	ValidationResult, AuditLog
Get rejected with a reason, leaving the word pending in its language	AuditLog
Keep the generated version so it can be compared with the corrected one	—

ContentGenerator

Type: control · **Superclass:** — · **Subclasses:** —

The service that turns words into draft content with the language model.

Responsibilities	Collaborators
Build batches of up to 100 words in one language, skipping words that already have approved content in it	Word, LanguageContent, GenerationBatch
Send the batch to the language model, using the official gloss as the meaning anchor	Language model (external)
Create a draft per word with senses, examples and translations	LanguageContent
Report the batch's progress and flag the words that failed so they can be retried	GenerationBatch

ContentValidator

Type: control · **Superclass:** — · **Subclasses:** —

The automatic quality check every draft goes through before an administrator can approve it.

Responsibilities	Collaborators
Check that the pinyin matches the hanzi	Word
Check that every example contains the word's hanzi	Example
Check that every translation is entirely in its language	Translation
Detect identical or nearly identical examples	Example

Record each failure with its reason	ValidationResult

VocabularyImporter

Type: control · **Superclass:** — · **Subclasses:** —

The service that loads vocabulary in bulk from a file.

Responsibilities	Collaborators
Accept only CSV or JSON files within the import limits	Import
Validate each row (required fields, level from 1 to 6) and record its errors by row number	ImportError
Create new words with pending content in both languages, without duplicating hanzi and pinyin	Word, LanguageContent
Return the summary: words created, duplicated and with errors	Import

3. Lists and decks

DeckList

Type: entity · **Superclass:** — · **Subclasses:** —

A group of decks: one of the six system HSK lists, or a list a user creates to organize their own decks.

Responsibilities	Collaborators
Know its name, its type (system or personal), its owner and, if it is a system list, its HSK level	User

Know its decks in order and change that order	ListDeck, Deck
Add and remove decks	ListDeck, Deck
Ensure its deck names are not repeated	Deck
When deleted, delete only the decks that belong to no other list	Deck
Reject changes from students and teachers when it is a system list	User

Deck

Type: entity · **Superclass:** — · **Subclasses:** —

A group of words studied together; it can belong to the system, to a user or to a course.

Responsibilities	Collaborators
Know its name, its type (system, personal or course) and its owner	User, Course
Know its words in order, and add or remove words	DeckWord, Word
If it is a system deck, accept only words of its level that are not in another deck of the same list	DeckList, Word
Compute its summary: total words, new words, reviews due today and percentage learned	RecognitionState, ProgressCalculator
Copy itself as an independent personal deck into one of the user's lists	DeckList

Tell whether a session can be started, that is, whether it has words	—
When deleted, remove itself from all its lists and discard its in-progress sessions	ListDeck, StudySession

4. Review and sessions

StudySession

Type: entity · **Superclass:** — · **Subclasses:** —

One study session on a deck, or on all due reviews, which can be paused and resumed on any device.

Responsibilities	Collaborators
Know its scope (scheduled, free review or all due reviews), its practice type, its writing mode and its status	—
Ensure a single in-progress session per user, deck and practice type	User, Deck
Get its queue when it starts and again when it is resumed	SessionQueueBuilder, SessionItem
Present the next card and, in writing, each character of the word in order, remembering the position so it can be resumed on any device	SessionItem, Word, WritingExercise
Record each answer, a grade or a written character, and request the state update except in free review	PracticeLog, RecognitionState, WritingState

Reinsert a card graded Hard after at least 3 other cards, and drop it after the third time	SessionItem
Undo the last grade	PracticeLog, RecognitionState
Discard itself while keeping the grades already recorded	PracticeLog
When completed, record the active day and return its summary	ActiveDay, ProgressCalculator

SessionQueueBuilder

Type: control · **Superclass:** — · **Subclasses:** —

The service that decides which words a session presents and in what order, so StudySession only has to run the session.

Responsibilities	Collaborators
Build a scheduled queue: the deck's due words first, then new words up to the daily limit	Deck, RecognitionState, WritingState
Apply the daily new-word limit across all decks, without counting free review	ProgressCalculator, User
Never present a word already studied as new in another deck	RecognitionState
Build a free review: every word of the deck in random order	Deck

Build the all-due-reviews queue: due words from all the user's decks, each once and with no new words	DeckList, Deck, RecognitionState
Recalculate a scheduled queue when the session is resumed, dropping words that are no longer due	SessionItem, RecognitionState

WritingExercise

Type: control · **Superclass:** — · **Subclasses:** —

One attempt at writing a character, stroke by stroke.

Responsibilities	Collaborators
Present a character in easy mode (faint template and next-stroke hint) or in hard mode (only pinyin and meaning)	Character, Word
Validate each stroke by shape, direction and order, within a tolerance margin	Character, Stroke data (external)
Count errors and, in hard mode, show a hint after three consecutive errors on the same stroke	—
Record "I don't know" and animate the whole character	Character
Return the result to the session: errors, hints, mode and whether it was "I don't know"	StudySession

ReviewState

Type: abstract · **Superclass:** — · **Subclasses:** RecognitionState, WritingState

What every review state shares: whose it is, when the item was last reviewed and when it is due again.

Responsibilities	Collaborators
Belong to exactly one user	User
Know the last and next review dates	—
Tell whether its review is due on a given local day	User
Leave to each subclass the decision of when its item counts as learned	—

RecognitionState

Type: entity · **Superclass:** ReviewState · **Subclasses:** —

How well a user recognizes one word, scheduled with FSRS; there is one per user and word, whatever the deck.

Responsibilities	Collaborators
Know the stability, difficulty and stage of a word for a user	Word
Tell whether the word is new, in learning or learned (stability of 21 days or more)	—
Update itself with a grade, delegating the calculation	FSRSScheduler
Restore its previous state when a grade is undone	PracticeLog

FSRSScheduler

Type: control · **Superclass:** — · **Subclasses:** —

The FSRS algorithm, isolated so it can be tuned or replaced without touching the rest of the system.

Responsibilities	Collaborators
Compute the new stability, difficulty and next review date from the current state and the grade	RecognitionState
Translate Easy into an FSRS success and Hard into an FSRS lapse	—
Read its parameters from configuration, without depending on the rest of the system	—

WritingState

Type: entity · **Superclass:** ReviewState · **Subclasses:** —

How well a user can write one character from memory; there is one per user and character, whatever the word.

Responsibilities	Collaborators
Know the interval and whether the character has been practiced in hard mode	Character
Tell whether the character is learned: interval of 21 days or more, practiced in hard mode	—
Update itself with the result of an exercise, delegating the calculation	WritingScheduler

WritingScheduler

Type: control · **Superclass:** — · **Subclasses:** —

HanziApp's own motor-memory algorithm for handwriting reviews.

Responsibilities	Collaborators
Compute the next interval from the errors, hints used and mode	WritingState
Give a shorter interval in easy mode than in hard mode for the same performance	—
Reset the interval when the user marks "I don't know"	—
Read its parameters from configuration, without depending on the rest of the system	—

PracticeLog

Type: abstract · **Superclass:** — · **Subclasses:** ReviewLog, WritingLog

The record of a single answer in a session: the history behind undo, the new-word limit and the statistics.

Responsibilities	Collaborators
Know the session and the moment the answer was given	StudySession
Know whether the answer affects scheduling, which is false in free review	—
For a grade, keep the previous state so it can be undone	RecognitionState

Provide the history for the new-word limit and the statistics	ProgressCalculator

5. Progress and motivation

ProgressCalculator

Type: control · **Superclass:** — · **Subclasses:** —

The service that turns review states and active days into the numbers the student sees; it stores nothing itself.

Responsibilities	Collaborators
Compute the current and longest streak from the active days	ActiveDay, User
Tell whether the streak is at risk today, for the 8:00 p.m. reminder	ActiveDay, User
Count distinct learned words, excluding deactivated ones	RecognitionState, Word
Count distinct learned characters	WritingState
Compute the progress percentage for each HSK level	RecognitionState, Word
Compute today's due reviews, separately for words and characters	RecognitionState, WritingState, User
Count the new words studied today in scheduled sessions, to enforce the daily limit	PracticeLog, User

Compute the minutes studied today against the daily goal	StudySession, User
Award a badge when a milestone is reached, only once per user	Badge, EarnedBadge

6. Courses

Course

Type: entity · **Superclass:** — · **Subclasses:** —

A group a teacher creates to assign decks to their students and follow their progress.

Responsibilities	Collaborators
Know its name, its teacher, its invitation code and whether it is active	User
Generate a unique 8-character code and regenerate it, invalidating the previous one	—
Enroll students with a valid code and remove them when they leave	Enrollment
Know its decks and assign to enrolled students the ones that have words	Deck
Present itself to each enrolled student as a read-only list containing the assigned decks	Deck, Enrollment
Provide each enrolled student's progress and the group averages	Enrollment, ProgressCalculator

Become inactive if its teacher loses the role	User
While inactive, reject new enrollments and edits, but keep its list for students already enrolled	Enrollment

Enrollment

Type: entity · **Superclass:** — · **Subclasses:** —

A student's membership in a course, including their consent to share progress with the teacher.

Responsibilities	Collaborators
Know the student, the course, the date and the consent to share progress	User, Course
Require the student to agree to share their progress before it is created	—
Expose to the teacher only the streak, active days, learned words and progress on assigned decks, never the email or the questionnaire	ProgressCalculator
When it ends, remove the course decks from the student's lists and discard their in-progress sessions	Course, StudySession

7. Traceability

Each class answers to a group of functional requirements and System Domain business rules. The Spanish name column matches the domain model.

Class	Spanish name	Type	RF	RN

User	Usuario	Entity	RF-01 to RF-23, RF-26 to RF-30, RF-82, RF-83, RF-87, RF-111 to RF-115	RN-15.1 to RN-15.4, RN-16.1 to RN-16.3, RN-17.1, RN-17.2, RN-18.1 to RN-18.3, RN-19.1, RN-19.2, RN-22.1 to RN-22.4, RN-23.1, RN-23.2, RN-53, RN-54
Word	Palabra	Entity	RF-31, RF-36 to RF-39, RF-42, RF-96 to RF-98, RF-107	RN-26.1, RN-26.2, RN-27.1, RN-27.2, RN-28
TranslatableElement	ElementoTraducible	Abstract	RF-36	RN-24
Dictionary	Diccionario	Control	RF-32 to RF-35, RF-40	RN-15.2, RN-15.3, RN-27.1
LanguageContent	ContenidoIdioma	Entity	RF-99, RF-104 to RF-106	RN-14, RN-24, RN-25.1 to RN-25.3
ContentGenerator	GeneradorContenido	Control	RF-102	RN-14, RN-51.1, RN-51.2
ContentValidator	ValidadorContenido	Control	RF-103	RN-25.3
VocabularyImporter	ImportadorVocabulario	Control	RF-100, RF-101	RN-26.1, RN-52.1, RN-52.2

DeckList	Lista	Entity	RF-43 to RF-46, RF-49, RF-110, new RFs for adding and removing a deck from a list	RN-03, RN-13, RN-29.1 to RN-29.3, RN-30.1, RN-33.1
Deck	Mazo	Entity	RF-41, RF-47, RF-48, RF-50 to RF-53, RF-108, RF-109	RN-03, RN-29.3, RN-30.2, RN-31.1 to RN-31.3, RN-32, RN-34
StudySession	SesionEstudio	Entity	RF-54, RF-55, RF-58, RF-60, RF-67, RF-71 to RF-74	RN-04.2, RN-12.1, RN-12.2, RN-36.1, RN-36.2, RN-42.1, RN-42.2, RN-43
SessionQueueBuilder	ConstructorColaSesion	Control	RF-59, RF-68 to RF-70, RF-72	RN-04.1, RN-04.3, RN-04.4, RN-12.3, RN-37.1, RN-37.3, RN-40, RN-41
WritingExercise	EjercicioEscritura	Control	RF-61 to RF-65	RN-39
ReviewState	EstadoRepaso	Abstract	RF-57, RF-66	RN-01, RN-02

RecognitionState	EstadoReconocimiento	Entity	RF-56, RF-57	RN-01, RN-05.1, RN-05.2, RN-06
FSRSScheduler	ProgramadorFSRS	Control	RF-57	RN-05.1, RN-05.2
WritingState	EstadoEscritura	Entity	RF-66	RN-02, RN-07, RN-38.3
WritingScheduler	ProgramadorEscritura	Control	RF-66	RN-38.1 to RN-38.3
PracticeLog	RegistroPractica	Abstract	RF-60, RF-71	RN-40, RN-42.1
ProgressCalculator	CalculadoraProgreso	Control	RF-59, RF-75 to RF-81, RF-84	RN-06 to RN-09, RN-10.1 to RN-10.3, RN-11, RN-37.1, RN-44, RN-45
Course	Curso	Entity	RF-88, RF-89, RF-91 to RF-94	RN-33.1, RN-33.2, RN-46.2, RN-47.1, RN-55
Enrollment	Inscripcion	Entity	RF-90, RF-95	RN-33.3, RN-43, RN-46.1, RN-47.1 to RN-47.3

Three requirements have no main class of their own: the landing page (RF-24 and RF-25) is presentation only, and usage metrics (RF-116) are computed with aggregate queries over sessions and users.

Collaborator classes without a card

Class	Spanish name
Role	Rol
AccessMethod	MetodoAcceso
DataPolicy	PoliticaDatos
PolicyAcceptance	AceptacionPolitica
LearnerProfile	PerfilAprendizaje
TeacherRequest	SolicitudProfesor
AuditLog	RegistroAuditoria
Character	Caracter
WordComposition	ComposicionPalabra
Sense (subclass of TranslatableElement)	Acepcion
Example (subclass of TranslatableElement)	Ejemplo
Translation	Traduccion

ValidationResult	ResultadoValidacion
ErrorReport	ReporteError
GenerationBatch	LoteGeneracion
Import	Importacion
ImportError	ErrorImportacion
ListDeck	ListaMazo
DeckWord	MazoPalabra
SessionItem	ItemSesion
ReviewLog (subclass of PracticeLog)	RegistroRepaso
WritingLog (subclass of PracticeLog)	RegistroEscritura
ActiveDay	DiaActivo
Badge	Insignia
EarnedBadge	LogroObtenido

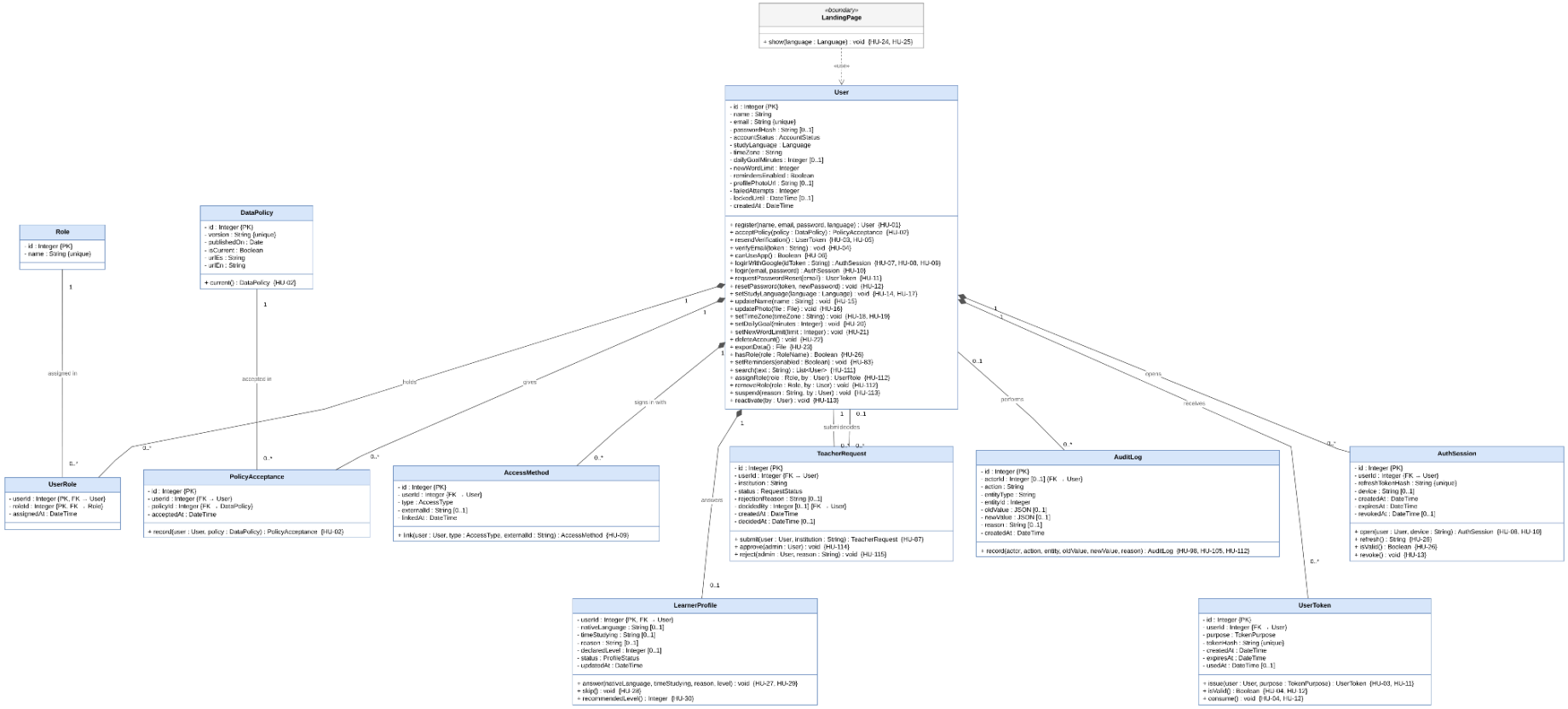
Class Diagram

HanziApp's class diagram has 52 classes, one of them an interface, and 22 enumerations, organized into eight packages. It builds on the domain model and the 22 CRC cards: each card's responsibilities become methods, and its collaborators become associations or dependencies. Class names match the English CRC cards.

An overview without detail comes first, followed by one diagram per package with attributes, methods, multiplicities, navigability and constraints. The Learning package is split into two diagrams, review states and study sessions, so neither becomes unreadable. Enumerations get a diagram of their own.

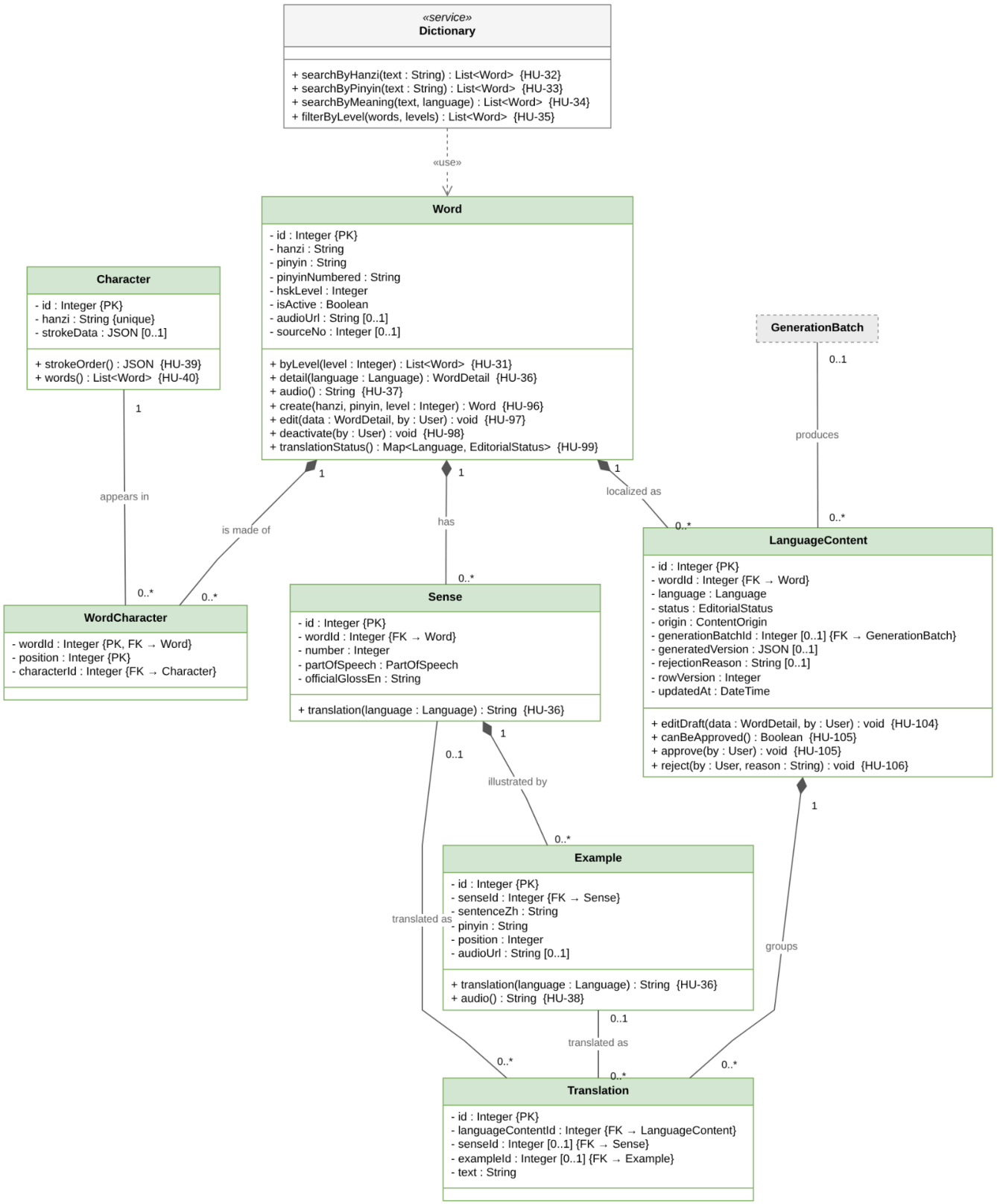
1. Accounts and access package

User holds the account's identity, settings and status; the other classes in the package record how the user signs in, what they accepted and what they requested.


«boundary»
LandingPage
+ show(language : Language) : void {HU-24, HU-25}
«use»
User
- id : Integer {PK}
- name : String
- email : String {unique}
- passwordHash : String [0..1]
- accountStatus : AccountStatus
- studyLanguage : Language
- timeZone : String
- dailyGoalMinutes : Integer [0..1]
- newWordLimit : Integer
- remindersEnabled : Boolean
- profilePhotoUrl : String [0..1]
- failedAttempts : Integer
- lockedUntil : DateTime [0..1]
- createdAt : DateTime
+ register(name, email, password, language) : User {HU-01}
+ acceptPolicy(policy : DataPolicy) : PolicyAcceptance {HU-02}
+ resendVerification() : UserToken {HU-03, HU-05}
+ verifyEmail(token : String) : void {HU-04}
+ canUseApp() : Boolean {HU-06}
+ loginWithGoogle(idToken : String) : AuthSession {HU-07, HU-08, HU-09}
+ login(email, password) : AuthSession {HU-10}
+ requestPasswordReset(email) : UserToken {HU-11}
+ resetPassword(token, newPassword) : void {HU-12}
+ setStudyLanguage(language : Language) : void {HU-14, HU-17}
+ updateName(name : String) : void {HU-15}
+ updatePhoto(file : File) : void {HU-16}
+ setTimeZone(timeZone : String) : void {HU-18, HU-19}
+ setDailyGoal(minutes : Integer) : void {HU-20}
+ setNewWordLimit(limit : Integer) : void {HU-21}
+ deleteAccount() : void {HU-22}
+ exportData() : File {HU-23}
+ hasRole(role : RoleName) : Boolean {HU-26}
+ setReminders(enabled : Boolean) : void {HU-83}
+ search(text : String) : List<User> {HU-111}
+ assignRole(role : Role, by : User) : UserRole {HU-112}
+ removeRole(role : Role, by : User) : void {HU-112}
+ suspend(reason : String, by : User) : void {HU-113}
+ reactivate(by : User) : void {HU-113}
Role
- id : Integer {PK}
- name : String {unique}
DataPolicy
- id : Integer {PK}
- version : String {unique}
- publishedOn : Date
- isCurrent : Boolean
- urlEs : String
- urlEn : String
+ current() : DataPolicy {HU-02}
UserRole
- userId : Integer {PK, FK → User}
- roleId : Integer {PK, FK → Role}
- assignedAt : DateTime
PolicyAcceptance
- id : Integer {PK}
- userId : Integer {FK → User}
- policyId : Integer {FK → DataPolicy}
- acceptedAt : DateTime
+ record(user : User, policy : DataPolicy) : PolicyAcceptance {HU-02}
AccessMethod
- id : Integer {PK}
- userId : Integer {FK → User}
- type : AccessType
- externalId : String [0..1]
- linkedAt : DateTime
+ link(user : User, type : AccessType, externalId : String) : AccessMethod {HU-09}
TeacherRequest
- id : Integer {PK}
- userId : Integer {FK → User}
- institution : String
- status : RequestStatus
- rejectionReason : String [0..1]
- decidedBy : Integer [0..1] {FK → User}
- createdAt : DateTime
- decidedAt : DateTime [0..1]
+ submit(user : User, institution : String) : TeacherRequest {HU-87}
+ approve(admin : User) : void {HU-114}
+ reject(admin : User, reason : String) : void {HU-115}
AuditLog
- id : Integer {PK}
- actorId : Integer [0..1] {FK → User}
- action : String
- entityType : String
- entityId : Integer
- oldValue : JSON [0..1]
- newValue : JSON [0..1]
- reason : String [0..1]
- createdAt : DateTime
+ record(actor, action, entity, oldValue, newValue, reason) : AuditLog {HU-98, HU-105, HU-112}
AuthSession
- id : Integer {PK}
- userId : Integer {FK → User}
- refreshTokenHash : String {unique}
- device : String [0..1]
- createdAt : DateTime
- expiresAt : DateTime
- revokedAt : DateTime [0..1]
+ open(user : User, device : String) : AuthSession {HU-08, HU-10}
+ refresh() : String {HU-28}
+ isValid() : Boolean {HU-76}
+ revoke() : void {HU-13}
LearnerProfile
- userId : Integer {PK, FK → User}
- nativeLanguage : String [0..1]
- timeStudying : String [0..1]
- reason : String [0..1]
- declaredLevel : Integer [0..1]
- status : ProfileStatus
- updatedAt : DateTime
+ answer(nativeLanguage, timeStudying, reason, level) : void {HU-27, HU-29}
+ skip() : void {HU-26}
+ recommendedLevel() : Integer {HU-30}
UserToken
- id : Integer {PK}
- userId : Integer {FK → User}
- purpose : TokenPurpose
- tokenHash : String {unique}
- createdAt : DateTime
- expiresAt : DateTime
- usedAt : DateTime [0..1]
+ issue(user : User, purpose : TokenPurpose) : UserToken {HU-03, HU-11}
+ isValid() : Boolean {HU-04, HU-12}
+ consume() : void {HU-04, HU-12}
assigned in
accepted in
holds
gives
signs in with
answers
submit decides
opens
receives
performs
1
0..*
0..1

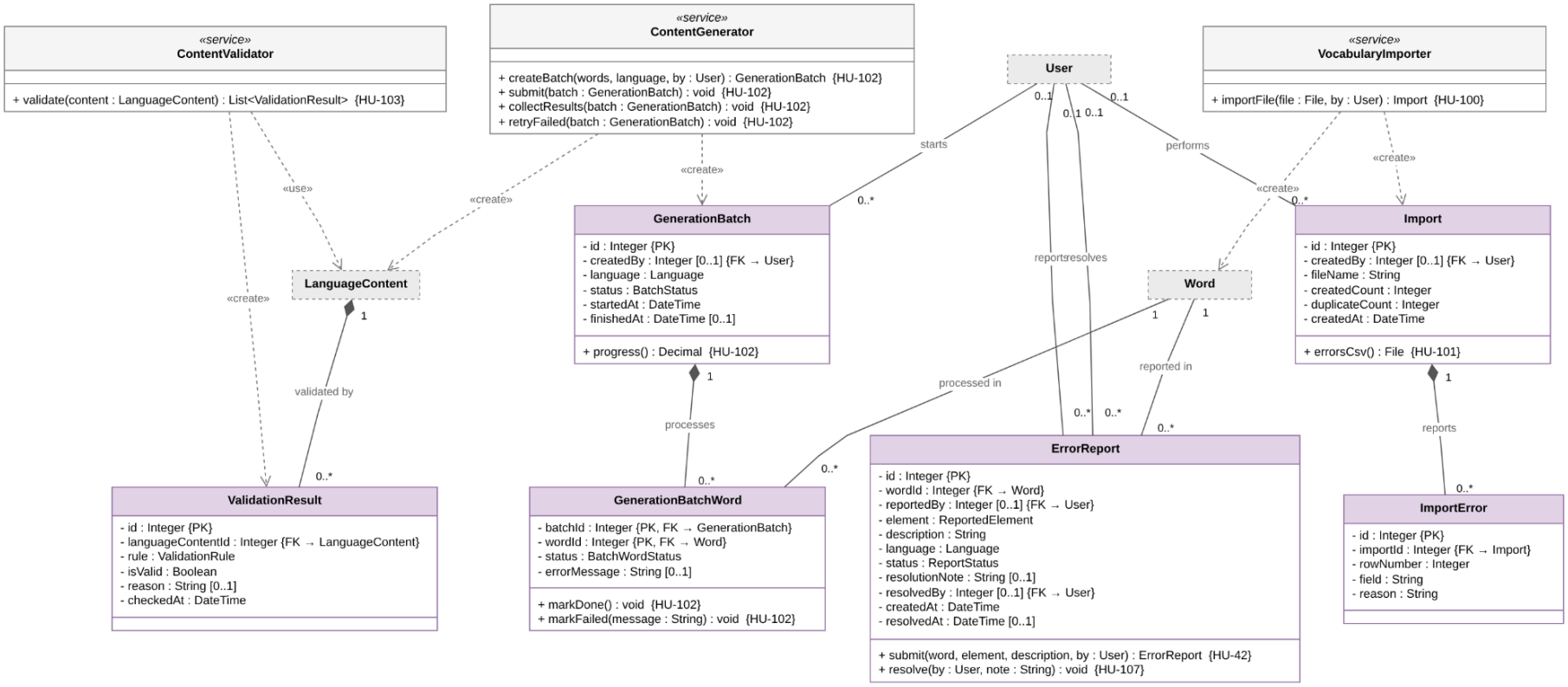
2. Content package

Content separates what is the same in every language (the word, its characters, its senses and its Chinese example sentences) from what changes per language (the translations, grouped in LanguageContent).


«service»
Dictionary
+ searchByHanzi(text : String) : List<Word> {HU-32}
+ searchByPinyin(text : String) : List<Word> {HU-33}
+ searchByMeaning(text, language) : List<Word> {HU-34}
+ filterByLevel(words, levels) : List<Word> {HU-35}
«use»
Word
- id : Integer {PK}
- hanzi : String
- pinyin : String
- pinyinNumbered : String
- hskLevel : Integer
- isActive : Boolean
- audioUrl : String [0..1]
- sourceNo : Integer [0..1]
+ byLevel(level : Integer) : List<Word> {HU-31}
+ detail(language : Language) : WordDetail {HU-36}
+ audio() : String {HU-37}
+ create(hanzi, pinyin, level : Integer) : Word {HU-96}
+ edit(data : WordDetail, by : User) : void {HU-97}
+ deactivate(by : User) : void {HU-98}
+ translationStatus() : Map<Language, EditorialStatus> {HU-99}
Character
- id : Integer {PK}
- hanzi : String {unique}
- strokeData : JSON [0..1]
+ strokeOrder() : JSON {HU-39}
+ words() : List<Word> {HU-40}
GenerationBatch
1
appears in
0..*
1
is made of
0..*
1
has
0..*
1
localized as
0..*
0..1
produces
0..*
WordCharacter
- wordId : Integer {PK, FK → Word}
- position : Integer {PK}
- characterId : Integer {FK → Character}
Sense
- id : Integer {PK}
- wordId : Integer {FK → Word}
- number : Integer
- partOfSpeech : PartOfSpeech
- officialGlossEn : String
+ translation(language : Language) : String {HU-36}
LanguageContent
- id : Integer {PK}
- wordId : Integer {FK → Word}
- language : Language
- status : EditorialStatus
- origin : ContentOrigin
- generationBatchId : Integer [0..1] {FK → GenerationBatch}
- generatedVersion : JSON [0..1]
- rejectionReason : String [0..1]
- rowVersion : Integer
- updatedAt : DateTime
+ editDraft(data : WordDetail, by : User) : void {HU-104}
+ canBeApproved() : Boolean {HU-105}
+ approve(by : User) : void {HU-105}
+ reject(by : User, reason : String) : void {HU-106}
1
0..1
illustrated by
0..*
1
groups
translated as
Example
- id : Integer {PK}
- senseId : Integer {FK → Sense}
- sentenceZh : String
- pinyin : String
- position : Integer
- audioUrl : String [0..1]
+ translation(language : Language) : String {HU-36}
+ audio() : String {HU-38}
0..1
translated as
0..*
0..*
0..*
Translation
- id : Integer {PK}
- languageContentId : Integer {FK → LanguageContent}
- senseId : Integer [0..1] {FK → Sense}
- exampleId : Integer [0..1] {FK → Example}
- text : String

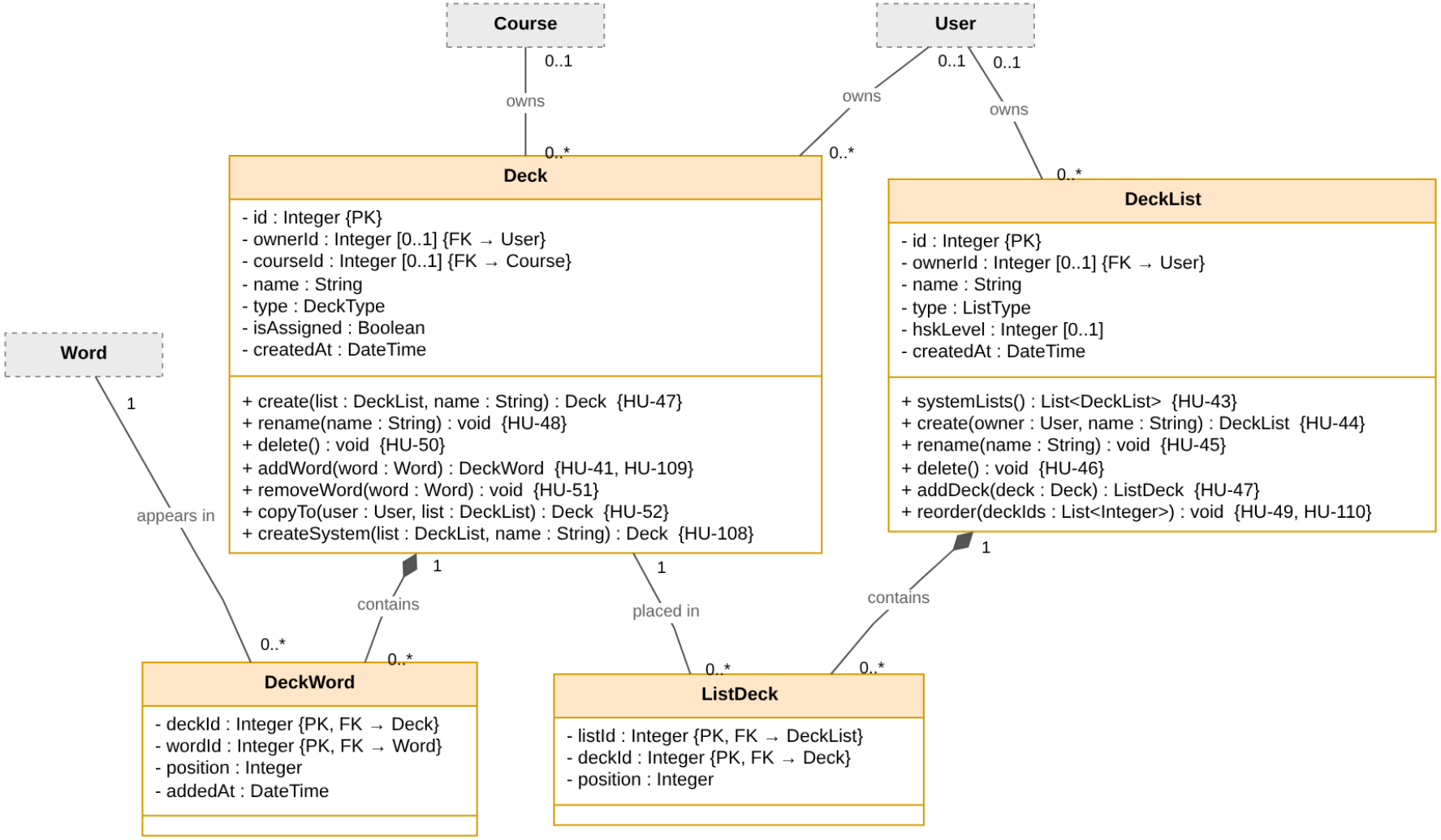
3. Content management package

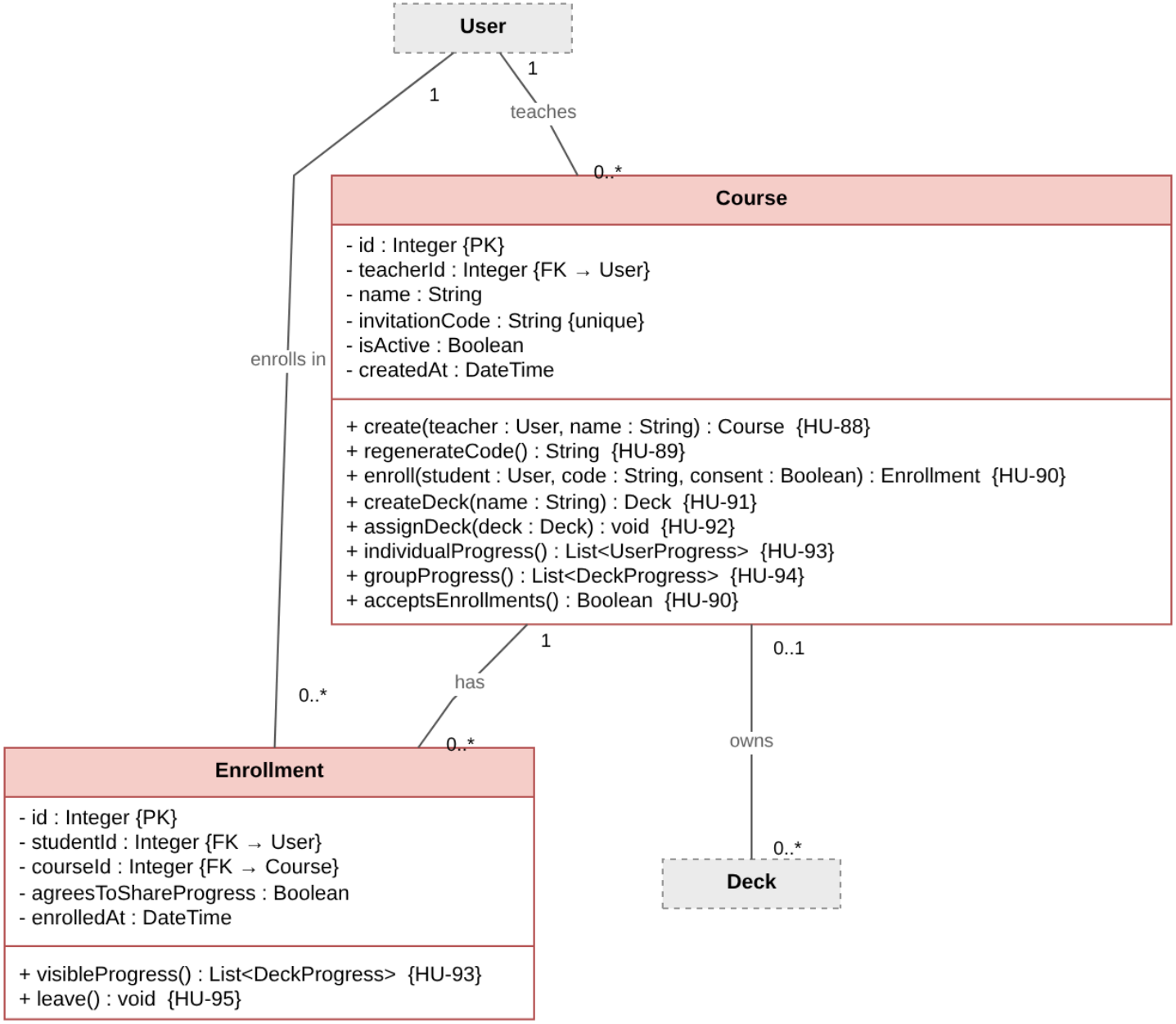
Content management gathers the administrator's tools: generating, validating and importing vocabulary, and handling user reports.


«service»
ContentValidator
+ validate(content : LanguageContent) : List<ValidationResult> {HU-103}
«service»
ContentGenerator
+ createBatch(words, language, by : User) : GenerationBatch {HU-102}
+ submit(batch : GenerationBatch) : void {HU-102}
+ collectResults(batch : GenerationBatch) : void {HU-102}
+ retryFailed(batch : GenerationBatch) : void {HU-102}
User
«service»
VocabularyImporter
+ importFile(file : File, by : User) : Import {HU-100}
«use»
«create»
«create»
«create»
starts
performs
«create»
«create»
0..1
0..1
0..1
0..1
0..*
0..*
LanguageContent
GenerationBatch
- id : Integer {PK}
- createdBy : Integer [0..1] {FK → User}
- language : Language
- status : BatchStatus
- startedAt : DateTime
- finishedAt : DateTime [0..1]
+ progress() : Decimal {HU-102}
reports
resolves
Word
Import
- id : Integer {PK}
- createdBy : Integer [0..1] {FK → User}
- fileName : String
- createdCount : Integer
- duplicateCount : Integer
- createdAt : DateTime
+ errorsCsv() : File {HU-101}
1
validated by
0..*
1
processes
0..*
1
1
processed in
0..*
1
reported in
0..*
0..*
0..*
1
reports
0..*
ValidationResult
- id : Integer {PK}
- languageContentId : Integer {FK → LanguageContent}
- rule : ValidationRule
- isValid : Boolean
- reason : String [0..1]
- checkedAt : DateTime
GenerationBatchWord
- batchId : Integer {PK, FK → GenerationBatch}
- wordId : Integer {PK, FK → Word}
- status : BatchWordStatus
- errorMessage : String [0..1]
+ markDone() : void {HU-102}
+ markFailed(message : String) : void {HU-102}
ErrorReport
- id : Integer {PK}
- wordId : Integer {FK → Word}
- reportedBy : Integer [0..1] {FK → User}
- element : ReportedElement
- description : String
- language : Language
- status : ReportStatus
- resolutionNote : String [0..1]
- resolvedBy : Integer [0..1] {FK → User}
- createdAt : DateTime
- resolvedAt : DateTime [0..1]
+ submit(word, element, description, by : User) : ErrorReport {HU-42}
+ resolve(by : User, note : String) : void {HU-107}
ImportError
- id : Integer {PK}
- importId : Integer {FK → Import}
- rowNumber : Integer
- field : String
- reason : String

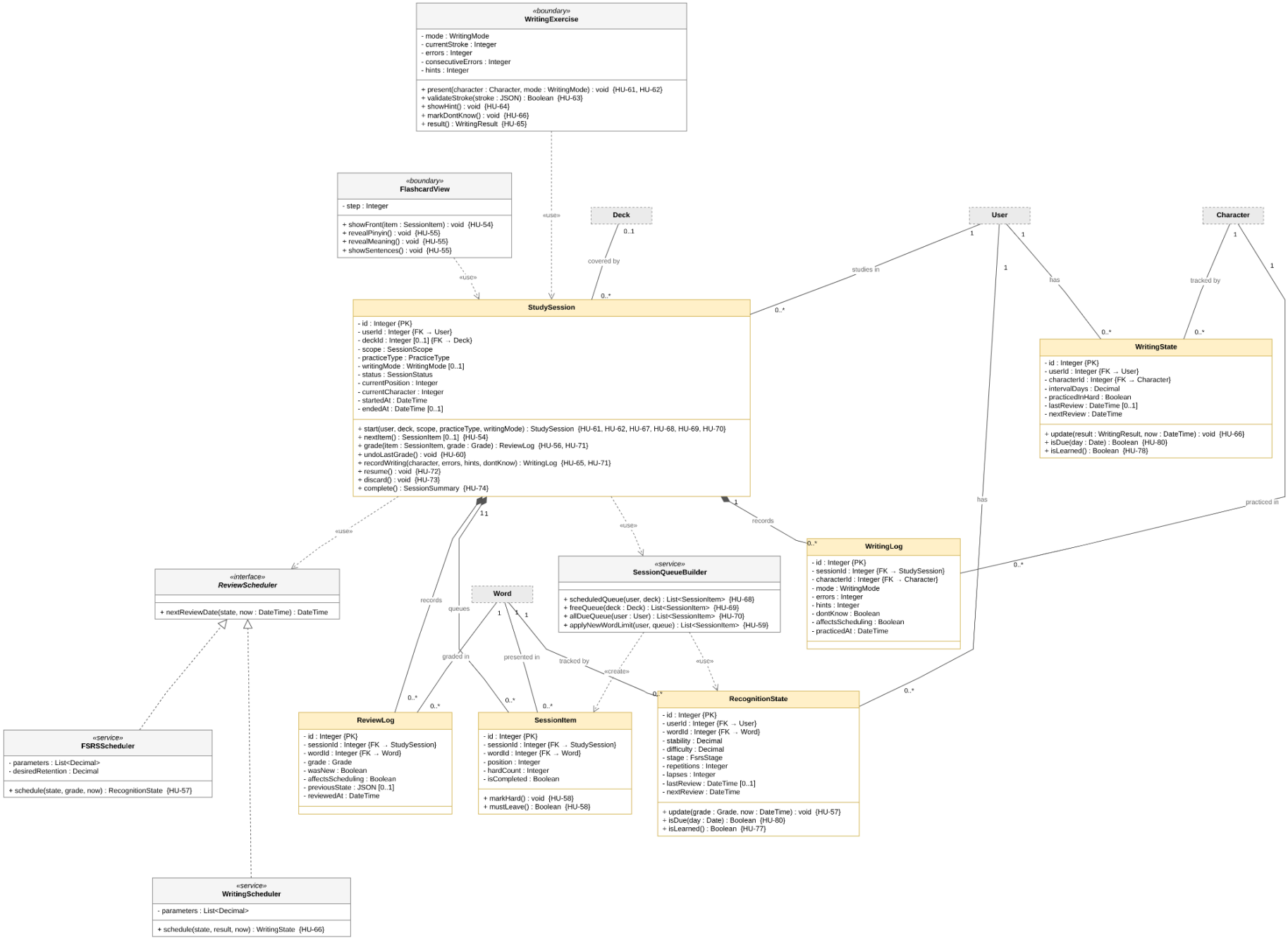
4. Organization and Courses packages

A DeckList contains decks and a Deck groups words; both relationships are many-to-many and store their order in intermediate classes.


Course
User
0..1
owns
0..*
0..1
0..1
owns
owns
0..*
0..*
Deck
- id : Integer {PK}
- ownerId : Integer [0..1] {FK → User}
- courseId : Integer [0..1] {FK → Course}
- name : String
- type : DeckType
- isAssigned : Boolean
- createdAt : DateTime
+ create(list : DeckList, name : String) : Deck {HU-47}
+ rename(name : String) : void {HU-48}
+ delete() : void {HU-50}
+ addWord(word : Word) : DeckWord {HU-41, HU-109}
+ removeWord(word : Word) : void {HU-51}
+ copyTo(user : User, list : DeckList) : Deck {HU-52}
+ createSystem(list : DeckList, name : String) : Deck {HU-108}
DeckList
- id : Integer {PK}
- ownerId : Integer [0..1] {FK → User}
- name : String
- type : ListType
- hskLevel : Integer [0..1]
- createdAt : DateTime
+ systemLists() : List<DeckList> {HU-43}
+ create(owner : User, name : String) : DeckList {HU-44}
+ rename(name : String) : void {HU-45}
+ delete() : void {HU-46}
+ addDeck(deck : Deck) : ListDeck {HU-47}
+ reorder(deckIds : List<Integer>) : void {HU-49, HU-110}
Word
1
appears in
0..*
1
contains
0..*
1
placed in
0..*
1
contains
0..*
DeckWord
- deckId : Integer {PK, FK → Deck}
- wordId : Integer {PK, FK → Word}
- position : Integer
- addedAt : DateTime
ListDeck
- listId : Integer {PK, FK → DeckList}
- deckId : Integer {PK, FK → Deck}
- position : Integer

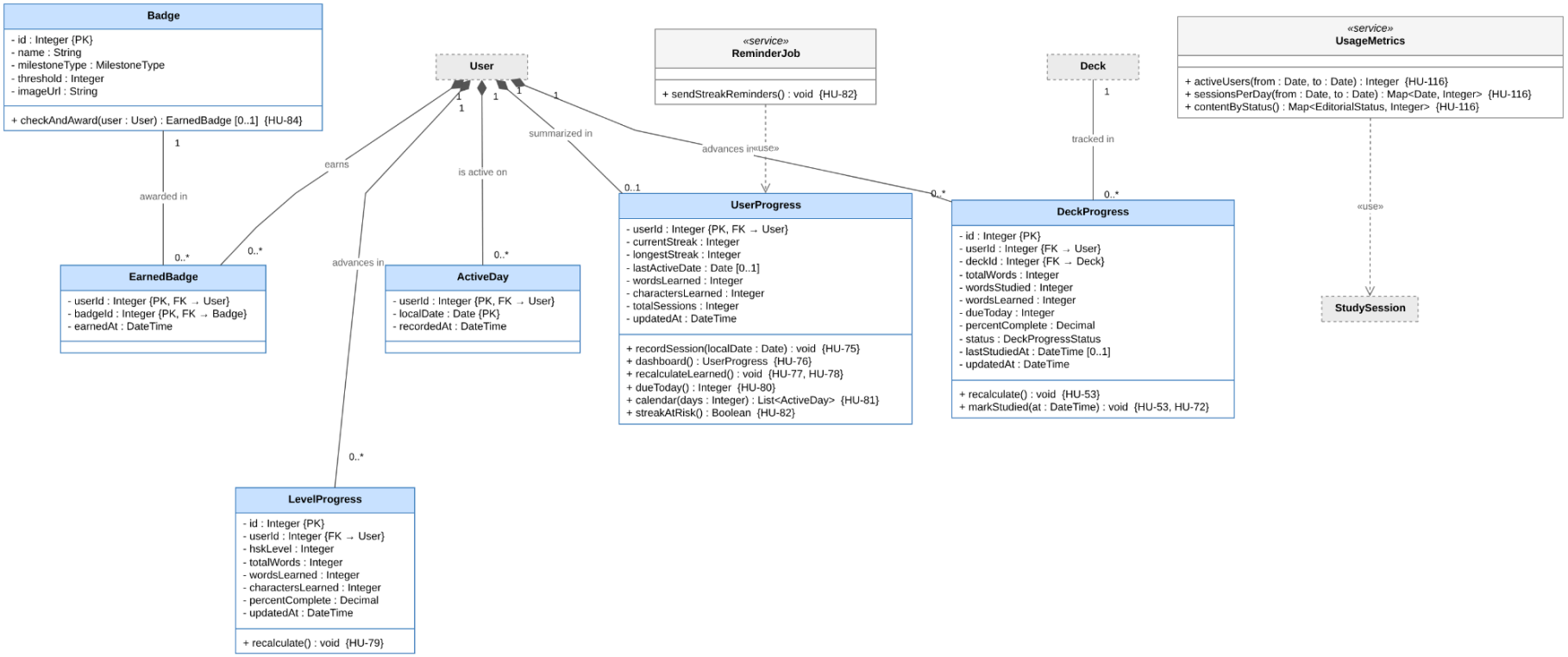

User
1
1
teaches
0..*
Course
- id : Integer {PK}
- teacherId : Integer {FK → User}
- name : String
- invitationCode : String {unique}
- isActive : Boolean
- createdAt : DateTime
+ create(teacher : User, name : String) : Course {HU-88}
+ regenerateCode() : String {HU-89}
+ enroll(student : User, code : String, consent : Boolean) : Enrollment {HU-90}
+ createDeck(name : String) : Deck {HU-91}
+ assignDeck(deck : Deck) : void {HU-92}
+ individualProgress() : List<UserProgress> {HU-93}
+ groupProgress() : List<DeckProgress> {HU-94}
+ acceptsEnrollments() : Boolean {HU-90}
enrolls in
0..*
1
has
0..*
0..1
owns
0..*
Enrollment
- id : Integer {PK}
- studentId : Integer {FK → User}
- courseId : Integer {FK → Course}
- agreesToShareProgress : Boolean
- enrolledAt : DateTime
+ visibleProgress() : List<DeckProgress> {HU-93}
+ leave() : void {HU-95}
Deck

5. Learning


«boundary»
WritingExercise
- mode : WritingMode
- currentStroke : Integer
- errors : Integer
- consecutiveErrors : Integer
- hints : Integer
+ present(character : Character, mode : WritingMode) : void {HU-61, HU-62}
+ validateStroke(stroke : JSON) : Boolean {HU-63}
+ showHint() : void {HU-64}
+ markDontKnow() : void {HU-66}
+ result() : WritingResult {HU-65}
«boundary»
FlashcardView
- step : Integer
+ showFront(item : SessionItem) : void {HU-54}
+ revealPinyin() : void {HU-55}
+ revealMeaning() : void {HU-55}
+ showSentences() : void {HU-55}
«use»
Deck
0..1
covered by
0..*
User
Character
studies in
has
tracked by
practiced in
StudySession
- id : Integer {PK}
- userId : Integer {FK → User}
- deckId : Integer [0..1] {FK → Deck}
- scope : SessionScope
- practiceType : PracticeType
- writingMode : WritingMode [0..1]
- status : SessionStatus
- currentPosition : Integer
- currentCharacter : Integer
- startedAt : DateTime
- endedAt : DateTime [0..1]
+ start(user, deck, scope, practiceType, writingMode) : StudySession {HU-61, HU-62, HU-67, HU-68, HU-69, HU-70}
+ nextItem() : SessionItem [0..1] {HU-54}
+ grade(item : SessionItem, grade : Grade) : ReviewLog {HU-56, HU-71}
+ undoLastGrade() : void {HU-60}
+ recordWriting(character, errors, hints, dontKnow) : WritingLog {HU-65, HU-71}
+ resume() : void {HU-72}
+ discard() : void {HU-73}
+ complete() : SessionSummary {HU-74}
WritingState
- id : Integer {PK}
- userId : Integer {FK → User}
- characterId : Integer {FK → Character}
- intervalDays : Decimal
- practicedInHard : Boolean
- lastReview : DateTime [0..1]
- nextReview : DateTime
+ update(result : WritingResult, now : DateTime) : void {HU-66}
+ isDue(day : Date) : Boolean {HU-80}
+ isLearned() : Boolean {HU-78}
records
«use»
«interface»
ReviewScheduler
+ nextReviewDate(state, now : DateTime) : DateTime
«service»
SessionQueueBuilder
+ scheduledQueue(user, deck) : List<SessionItem> {HU-68}
+ freeQueue(deck : Deck) : List<SessionItem> {HU-69}
+ allDueQueue(user : User) : List<SessionItem> {HU-70}
+ applyNewWordLimit(user, queue) : List<SessionItem> {HU-59}
WritingLog
- id : Integer {PK}
- sessionId : Integer {FK → StudySession}
- characterId : Integer {FK → Character}
- mode : WritingMode
- errors : Integer
- hints : Integer
- dontKnow : Boolean
- affectsScheduling : Boolean
- practicedAt : DateTime
Word
queues
graded in
presented in
tracked by
«create»
«use»
ReviewLog
- id : Integer {PK}
- sessionId : Integer {FK → StudySession}
- wordId : Integer {FK → Word}
- grade : Grade
- wasNew : Boolean
- affectsScheduling : Boolean
- previousState : JSON [0..1]
- reviewedAt : DateTime
SessionItem
- id : Integer {PK}
- sessionId : Integer {FK → StudySession}
- wordId : Integer {FK → Word}
- position : Integer
- hardCount : Integer
- isCompleted : Boolean
+ markHard() : void {HU-58}
+ mustLeave() : Boolean {HU-58}
RecognitionState
- id : Integer {PK}
- userId : Integer {FK → User}
- wordId : Integer {FK → Word}
- stability : Decimal
- difficulty : Decimal
- stage : FsrsStage
- repetitions : Integer
- lapses : Integer
- lastReview : DateTime [0..1]
- nextReview : DateTime
+ update(grade : Grade, now : DateTime) : void {HU-57}
+ isDue(day : Date) : Boolean {HU-80}
+ isLearned() : Boolean {HU-77}
«service»
FSRSScheduler
- parameters : List<Decimal>
- desiredRetention : Decimal
+ schedule(state, grade, now) : RecognitionState {HU-57}
«service»
WritingScheduler
- parameters : List<Decimal>
+ schedule(state, result, now) : WritingState {HU-66}

6. Progress package

Progress keeps only two records, active days and earned badges; every other number is computed by ProgressCalculator.


Badge
- id : Integer {PK}
- name : String
- milestoneType : MilestoneType
- threshold : Integer
- imageUrl : String
+ checkAndAward(user : User) : EarnedBadge [0..1] {HU-84}
User
«service»
ReminderJob
+ sendStreakReminders() : void {HU-82}
Deck
«service»
UsageMetrics
+ activeUsers(from : Date, to : Date) : Integer {HU-116}
+ sessionsPerDay(from : Date, to : Date) : Map<Date, Integer> {HU-116}
+ contentByStatus() : Map<EditorialStatus, Integer> {HU-116}
awarded in
earns
is active on
summarized in
advances in
«use»
tracked in
«use»
EarnedBadge
- userId : Integer {PK, FK → User}
- badgeId : Integer {PK, FK → Badge}
- earnedAt : DateTime
ActiveDay
- userId : Integer {PK, FK → User}
- localDate : Date {PK}
- recordedAt : DateTime
UserProgress
- userId : Integer {PK, FK → User}
- currentStreak : Integer
- longestStreak : Integer
- lastActiveDate : Date [0..1]
- wordsLearned : Integer
- charactersLearned : Integer
- totalSessions : Integer
- updatedAt : DateTime
+ recordSession(localDate : Date) : void {HU-75}
+ dashboard() : UserProgress {HU-76}
+ recalculateLearned() : void {HU-77, HU-78}
+ dueToday() : Integer {HU-80}
+ calendar(days : Integer) : List<ActiveDay> {HU-81}
+ streakAtRisk() : Boolean {HU-82}
DeckProgress
- id : Integer {PK}
- userId : Integer {FK → User}
- deckId : Integer {FK → Deck}
- totalWords : Integer
- wordsStudied : Integer
- wordsLearned : Integer
- dueToday : Integer
- percentComplete : Decimal
- status : DeckProgressStatus
- lastStudiedAt : DateTime [0..1]
- updatedAt : DateTime
+ recalculate() : void {HU-53}
+ markStudied(at : DateTime) : void {HU-53, HU-72}
StudySession
LevelProgress
- id : Integer {PK}
- userId : Integer {FK → User}
- hskLevel : Integer
- totalWords : Integer
- wordsLearned : Integer
- charactersLearned : Integer
- percentComplete : Decimal
- updatedAt : DateTime
+ recalculate() : void {HU-79}
1
0..*
0..1

7. Enumerations

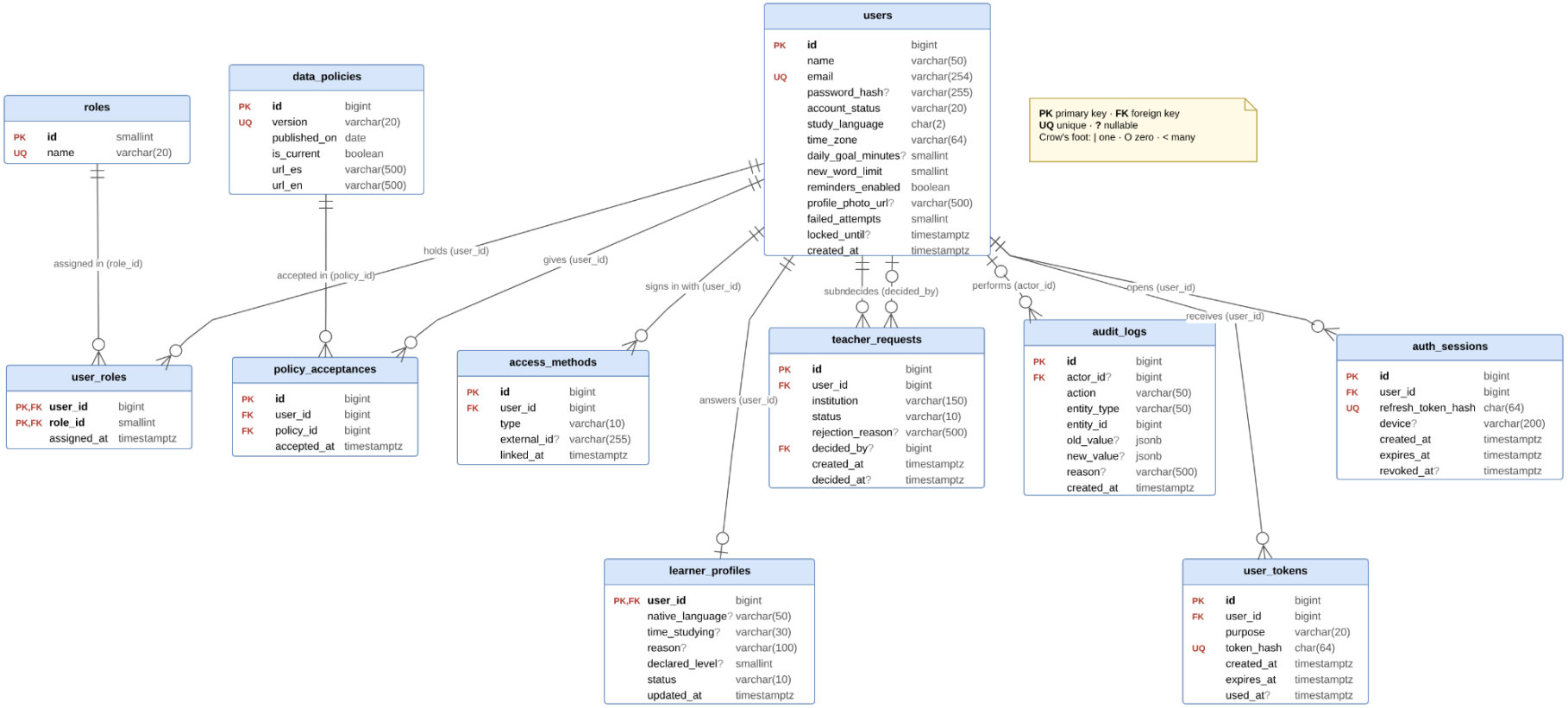

«enumeration»
AccountStatus
pending
verified
suspended
«enumeration»
AccessType
email
google
«enumeration»
Language
es
en
«enumeration»
ProfileStatus
pending
skipped
completed
«enumeration»
RequestStatus
pending
approved
rejected
«enumeration»
TokenPurpose
email_verification
password_reset
«enumeration»
PartOfSpeech
noun
verb
adjective
adverb
pronoun
numeral
measure_word
preposition
conjunction
particle
interjection
expression
«enumeration»
EditorialStatus
pending
draft
approved
rejected
«enumeration»
ContentOrigin
ai
manual
imported
«enumeration»
ValidationRule
pinyin_mismatch
hanzi_missing
wrong_language
repeated_examples
«enumeration»
ReportedElement
pinyin
sense
example
translation
audio
«enumeration»
ReportStatus
pending
resolved
«enumeration»
BatchStatus
in_progress
completed
with_errors
«enumeration»
BatchWordStatus
pending
done
failed
«enumeration»
ListType
system
personal
«enumeration»
DeckType
system
personal
course
«enumeration»
Grade
easy
hard
«enumeration»
FsrsStage
new
learning
review
relearning
«enumeration»
SessionScope
scheduled
free
all_due
«enumeration»
PracticeType
flashcards
writing
both
«enumeration»
WritingMode
easy
hard
«enumeration»
SessionStatus
in_progress
completed
discarded
«enumeration»
MilestoneType
streak
hsk_level
«enumeration»
DeckProgressStatus
in_progress
completed
«datatype»
SessionSummary
- reviewed : Integer
- easy : Integer
- hard : Integer
- charactersWritten : Integer
- writingErrors : Integer
- hintsUsed : Integer
- streakIncreased : Boolean
«datatype»
WritingResult
- errors : Integer
- hints : Integer
- dontKnow : Boolean
- mode : WritingMode
«datatype»
WordDetail
- hanzi : String
- pinyin : String
- senses : List<Sense>
- examples : List<Example>
- hskLevel : Integer

Grade has only two values, EASY and HARD, as the professor requested. FSRSScheduler maps them to the FSRS grades Good and Again (RN-05.1, RN-05.2).

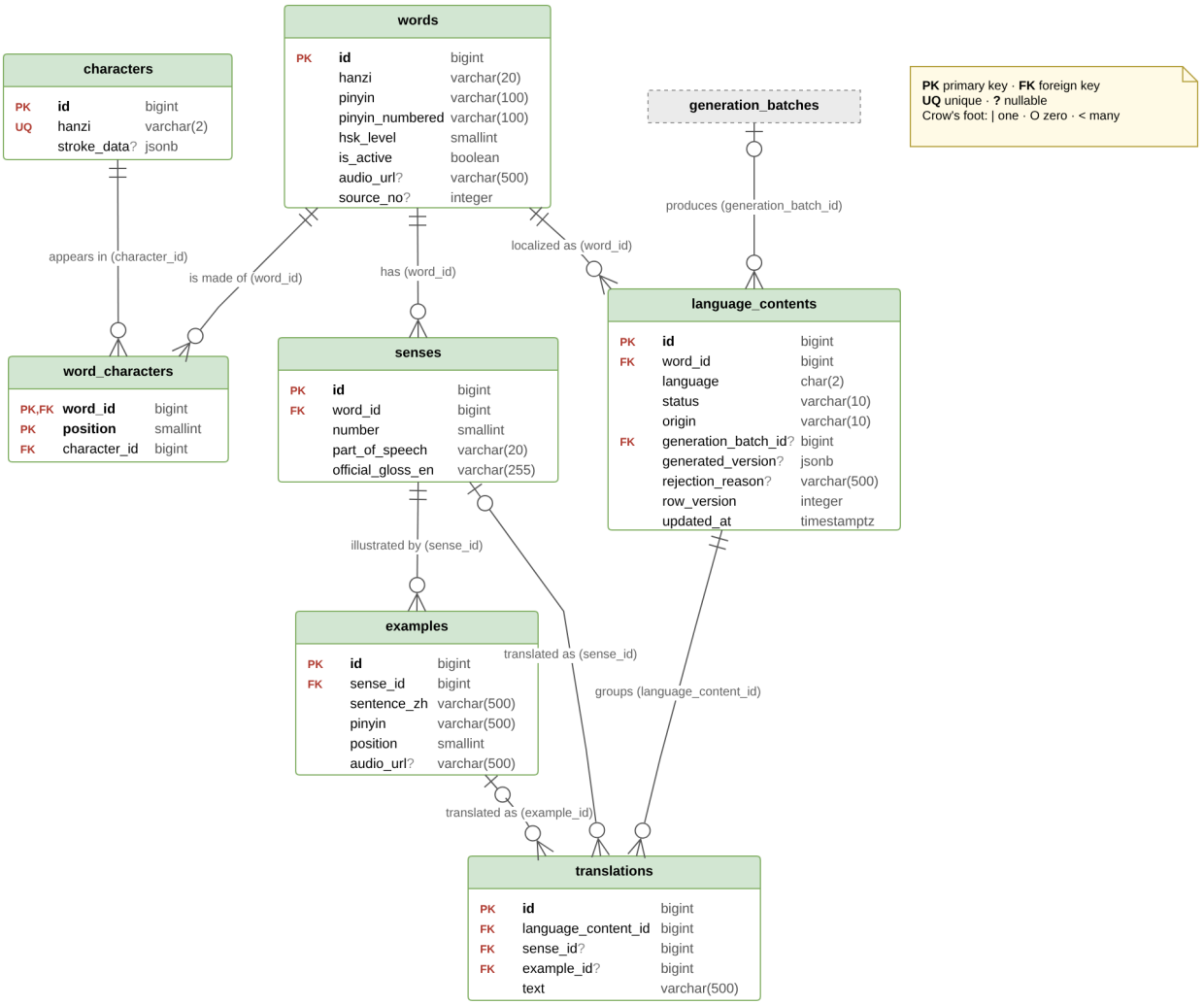
Relational Database Model

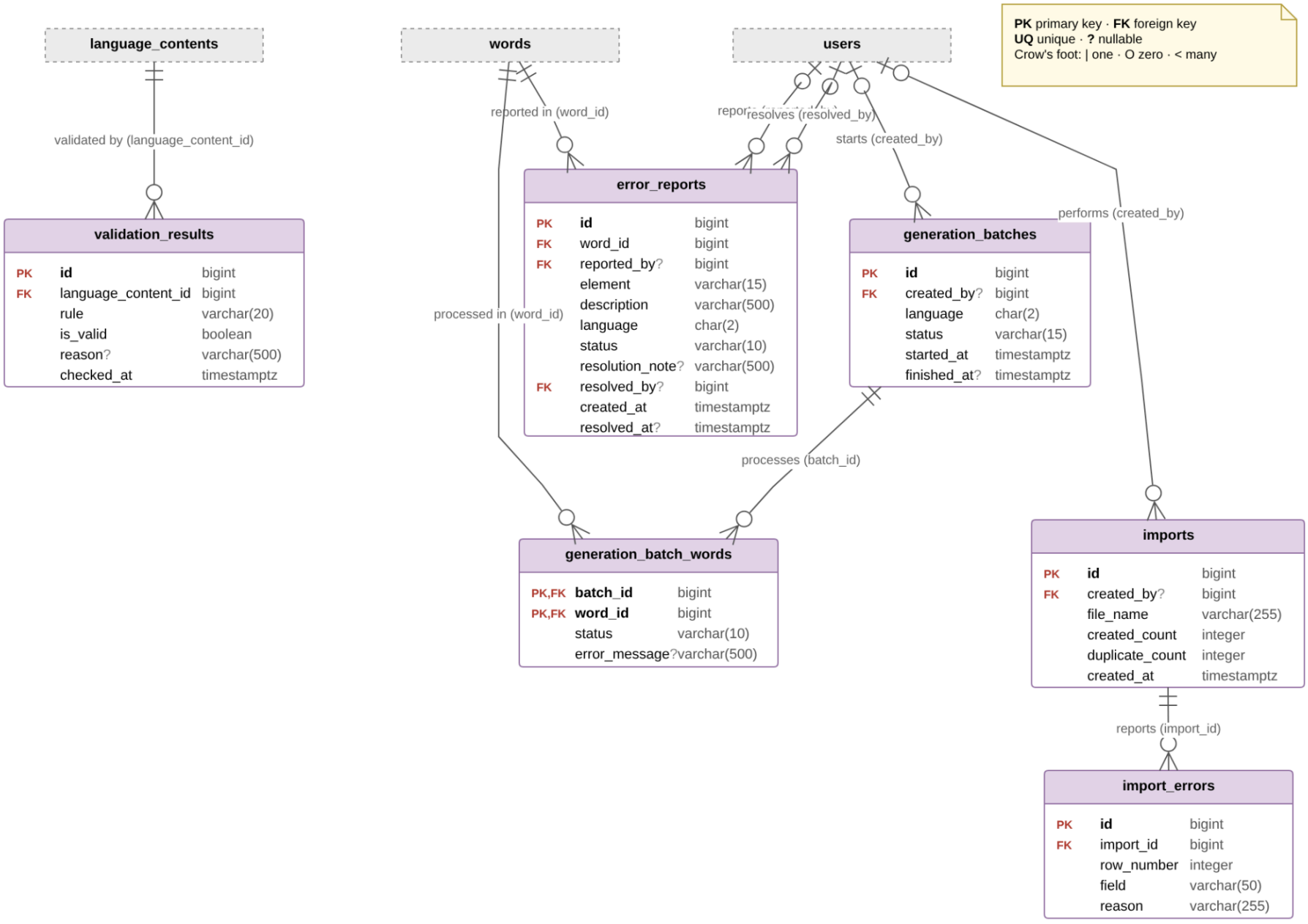
1. Accounts and access

Eleven tables hold the account, how the user signs in, what they accepted and the administrative trail.

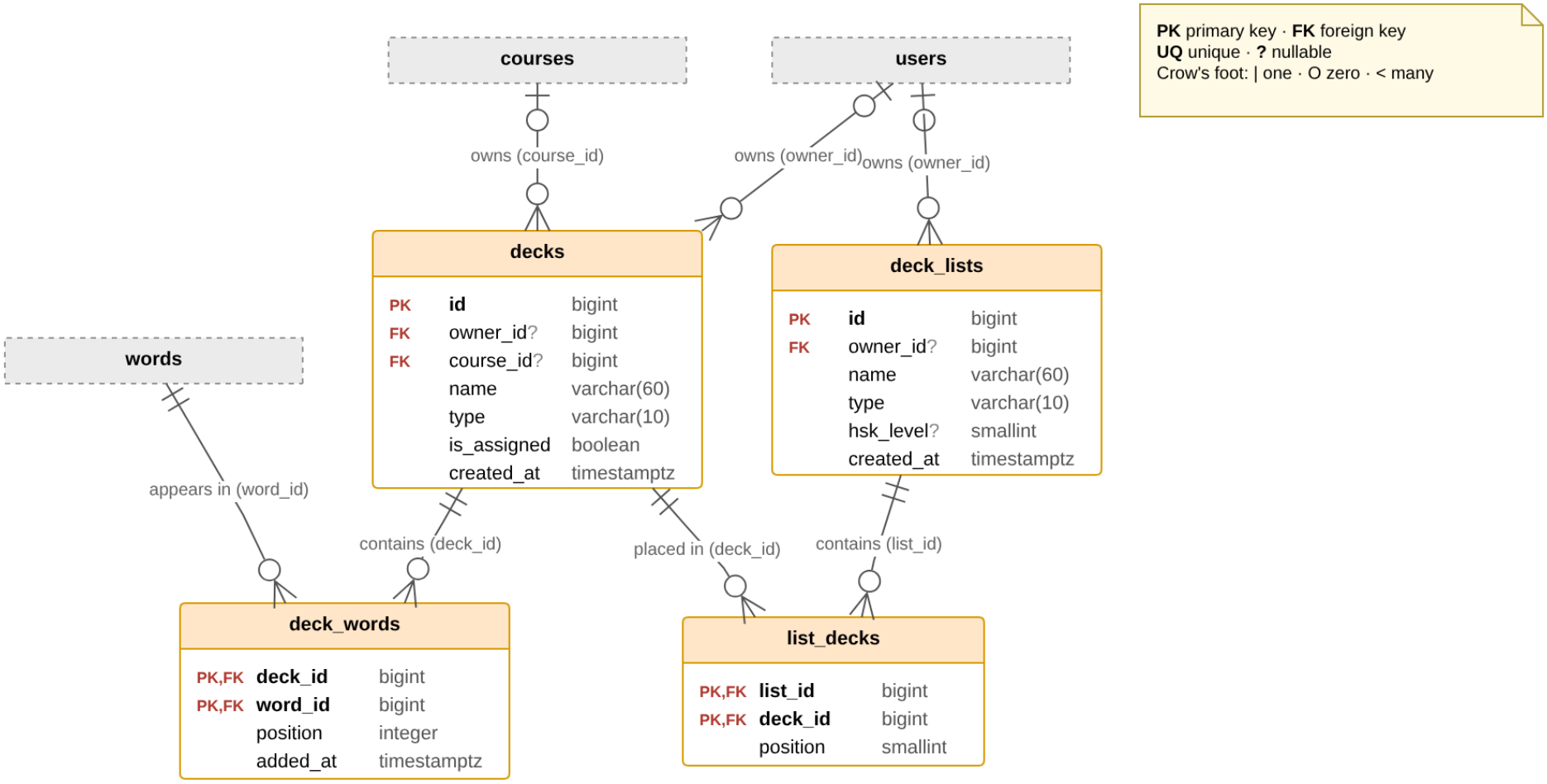

users
PK id bigint
name varchar(50)
UQ email varchar(254)
password_hash? varchar(255)
account_status varchar(20)
study_language char(2)
time_zone varchar(64)
daily_goal_minutes? smallint
new_word_limit smallint
reminders_enabled boolean
profile_photo_url? varchar(500)
failed_attempts smallint
locked_until? timestamptz
created_at timestamptz
PK primary key · FK foreign key
UQ unique · ? nullable
Crow's foot: | one · O zero · < many
roles
PK id smallint
UQ name varchar(20)
data_policies
PK id bigint
UQ version varchar(20)
published_on date
is_current boolean
url_es varchar(500)
url_en varchar(500)
assigned in (role_id)
accepted in (policy_id)
holds (user_id)
gives (user_id)
signs in with (user_id)
answers (user_id)
subndecides (decided_by)
performs (actor_id)
opens (user_id)
receives (user_id)
user_roles
PK,FK user_id bigint
PK,FK role_id smallint
assigned_at timestamptz
policy_acceptances
PK id bigint
FK user_id bigint
FK policy_id bigint
accepted_at timestamptz
access_methods
PK id bigint
FK user_id bigint
type varchar(10)
external_id? varchar(255)
linked_at timestamptz
teacher_requests
PK id bigint
FK user_id bigint
institution varchar(150)
status varchar(10)
rejection_reason? varchar(500)
FK decided_by? bigint
created_at timestamptz
decided_at? timestamptz
audit_logs
PK id bigint
FK actor_id? bigint
action varchar(50)
entity_type varchar(50)
entity_id bigint
old_value? jsonb
new_value? jsonb
reason? varchar(500)
created_at timestamptz
auth_sessions
PK id bigint
FK user_id bigint
UQ refresh_token_hash char(64)
device? varchar(200)
created_at timestamptz
expires_at timestamptz
revoked_at? timestamptz
learner_profiles
PK,FK user_id bigint
native_language? varchar(50)
time_studying? varchar(30)
reason? varchar(100)
declared_level? smallint
status varchar(10)
updated_at timestamptz
user_tokens
PK id bigint
FK user_id bigint
purpose varchar(20)
UQ token_hash char(64)
created_at timestamptz
expires_at timestamptz
used_at? timestamptz

2. Content and content management

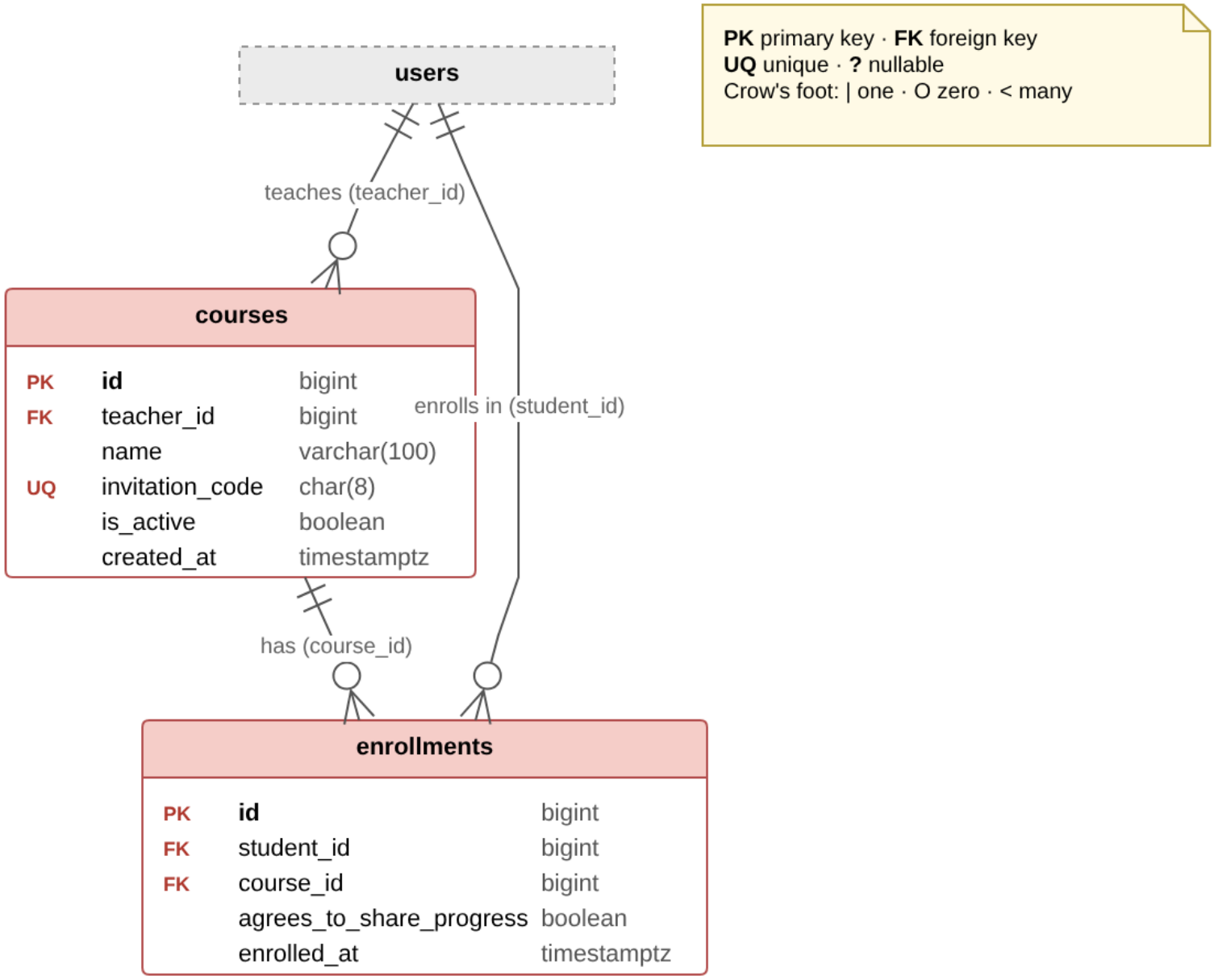



language_contents
validated by (language_content_id)
validation_results
PK id bigint
FK language_content_id bigint
rule varchar(20)
is_valid boolean
reason? varchar(500)
checked_at timestamptz
words
users
PK primary key · FK foreign key
UQ unique · ? nullable
Crow's foot: | one · O zero · < many
reported in (word_id)
resolves (resolved_by)
starts (created_by)
performs (created_by)
processed in (word_id)
processes (batch_id)
error_reports
PK id bigint
FK word_id bigint
FK reported_by? bigint
element varchar(15)
description varchar(500)
language char(2)
status varchar(10)
resolution_note? varchar(500)
FK resolved_by? bigint
created_at timestamptz
resolved_at? timestamptz
generation_batches
PK id bigint
FK created_by? bigint
language char(2)
status varchar(15)
started_at timestamptz
finished_at? timestamptz
generation_batch_words
PK,FK batch_id bigint
PK,FK word_id bigint
status varchar(10)
error_message? varchar(500)
imports
PK id bigint
FK created_by? bigint
file_name varchar(255)
created_count integer
duplicate_count integer
created_at timestamptz
reports (import_id)
import_errors
PK id bigint
FK import_id bigint
row_number integer
field varchar(50)
reason varchar(255)

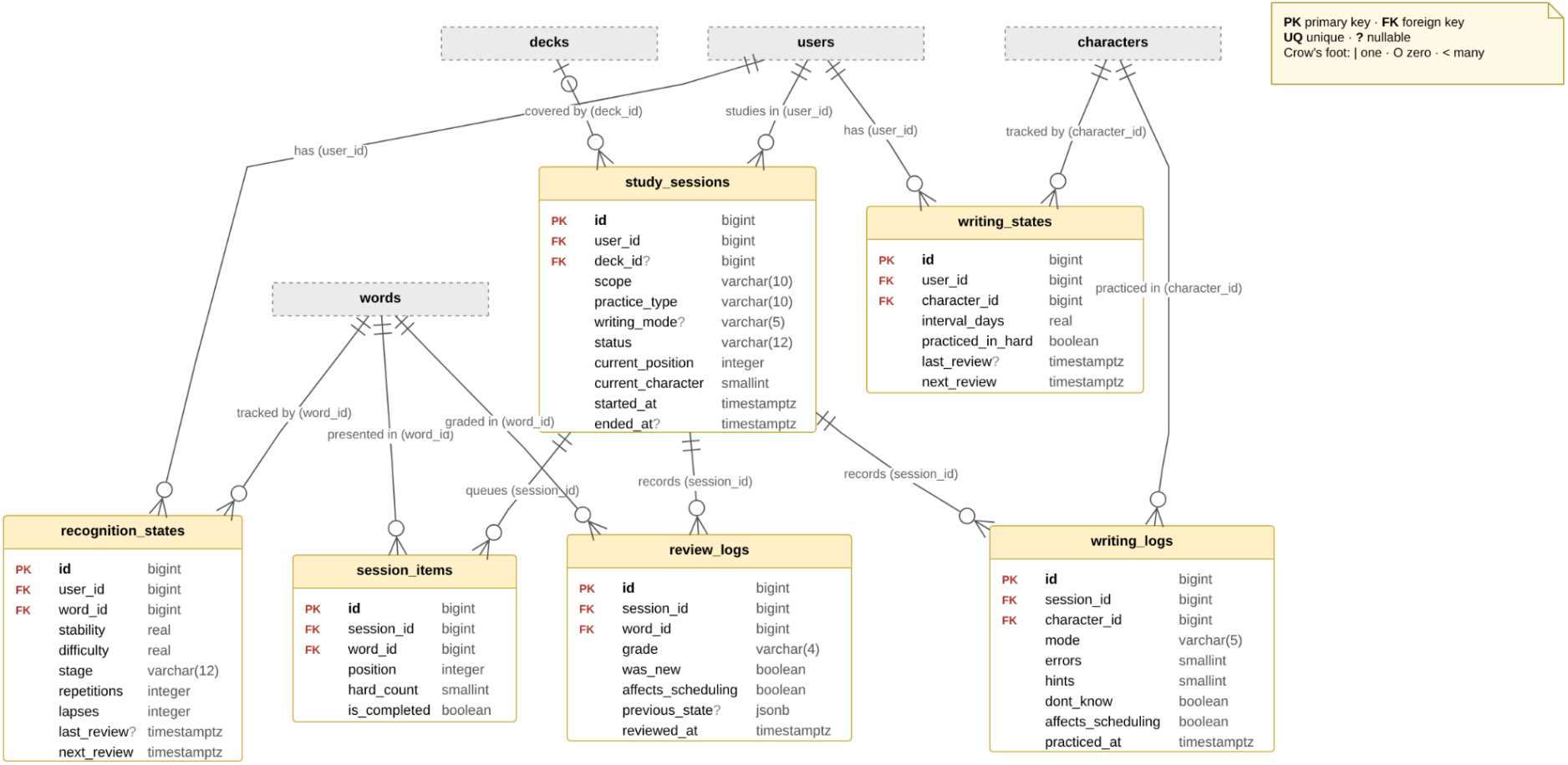
3. Organization and courses


courses
users
owns (course_id)
owns (owner_id)
owns (owner_id)
decks
PK id bigint
FK owner_id? bigint
FK course_id? bigint
name varchar(60)
type varchar(10)
is_assigned boolean
created_at timestamptz
deck_lists
PK id bigint
FK owner_id? bigint
name varchar(60)
type varchar(10)
hsk_level? smallint
created_at timestamptz
words
appears in (word_id)
contains (deck_id)
placed in (deck_id)
contains (list_id)
deck_words
PK,FK deck_id bigint
PK,FK word_id bigint
position integer
added_at timestamptz
list_decks
PK,FK list_id bigint
PK,FK deck_id bigint
position smallint
PK primary key · FK foreign key
UQ unique · ? nullable
Crow's foot: | one · O zero · < many

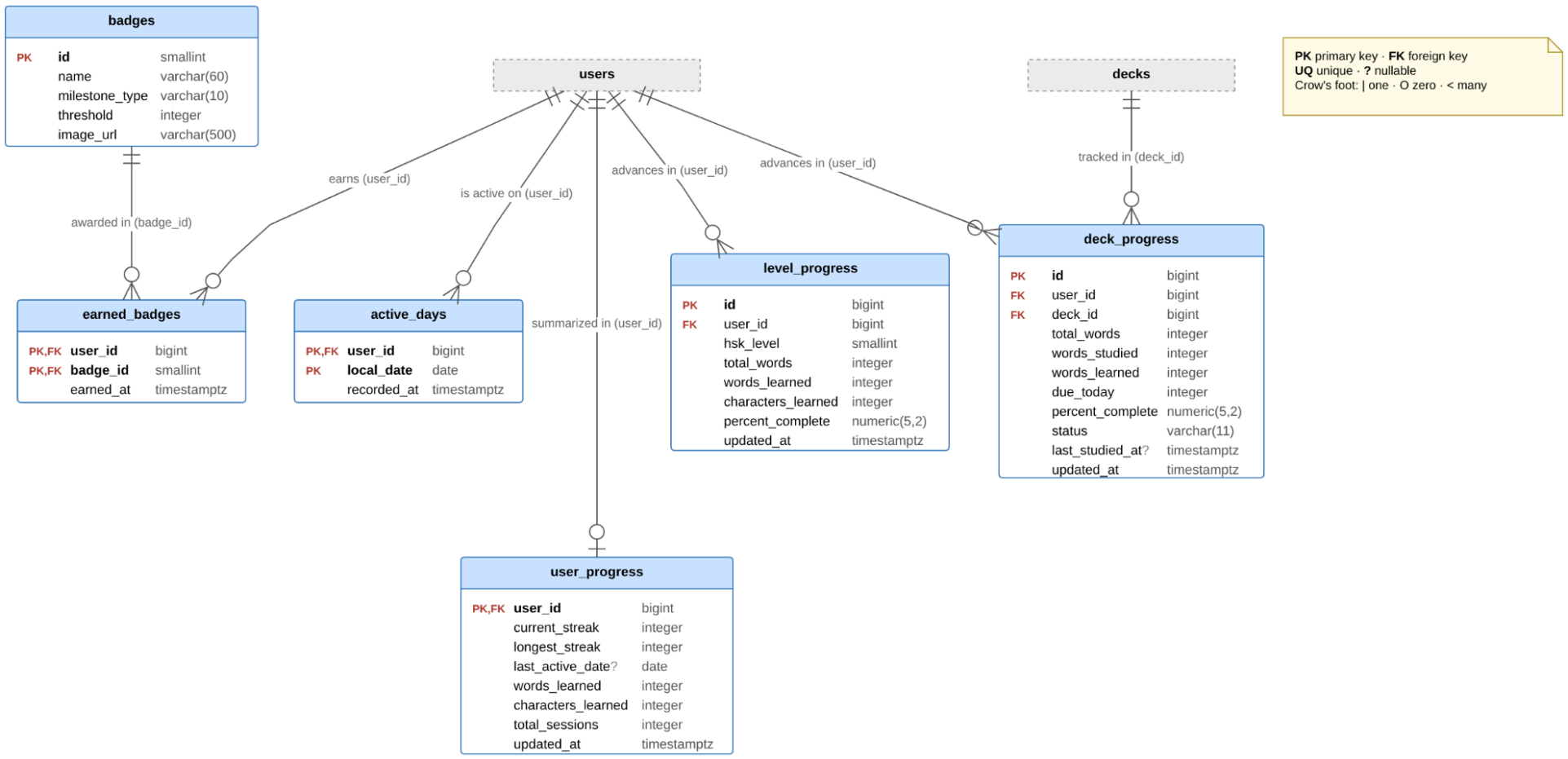
4. Courses


users
teaches (teacher_id)
courses
PK id bigint
FK teacher_id bigint
name varchar(100)
UQ invitation_code char(8)
is_active boolean
created_at timestamptz
enrolls in (student_id)
has (course_id)
enrollments
PK id bigint
FK student_id bigint
FK course_id bigint
agrees_to_share_progress boolean
enrolled_at timestamptz
PK primary key · FK foreign key
UQ unique · ? nullable
Crow's foot: | one · O zero · < many

5. Learning


PK primary key · FK foreign key
UQ unique · ? nullable
Crow's foot: | one · O zero · < many
decks
users
characters
covered by (deck_id)
studies in (user_id)
has (user_id)
has (user_id)
tracked by (character_id)
study_sessions
PK id bigint
FK user_id bigint
FK deck_id? bigint
scope varchar(10)
practice_type varchar(10)
writing_mode? varchar(5)
status varchar(12)
current_position integer
current_character smallint
started_at timestamptz
ended_at? timestamptz
writing_states
PK id bigint
FK user_id bigint
FK character_id bigint
interval_days real
practiced_in_hard boolean
last_review? timestamptz
next_review timestamptz
practiced in (character_id)
words
tracked by (word_id)
presented in (word_id)
graded in (word_id)
queues (session_id)
records (session_id)
records (session_id)
recognition_states
PK id bigint
FK user_id bigint
FK word_id bigint
stability real
difficulty real
stage varchar(12)
repetitions integer
lapses integer
last_review? timestamptz
next_review timestamptz
session_items
PK id bigint
FK session_id bigint
FK word_id bigint
position integer
hard_count smallint
is_completed boolean
review_logs
PK id bigint
FK session_id bigint
FK word_id bigint
grade varchar(4)
was_new boolean
affects_scheduling boolean
previous_state? jsonb
reviewed_at timestamptz
writing_logs
PK id bigint
FK session_id bigint
FK character_id bigint
mode varchar(5)
errors smallint
hints smallint
dont_know boolean
affects_scheduling boolean
practiced_at timestamptz

6. Progress


badges
PK id smallint
name varchar(60)
milestone_type varchar(10)
threshold integer
image_url varchar(500)
users
decks
PK primary key · FK foreign key
UQ unique · ? nullable
Crow's foot: | one · O zero · < many
tracked in (deck_id)
awarded in (badge_id)
earns (user_id)
is active on (user_id)
advances in (user_id)
advances in (user_id)
summarized in (user_id)
deck_progress
PK id bigint
FK user_id bigint
FK deck_id bigint
total_words integer
words_studied integer
words_learned integer
due_today integer
percent_complete numeric(5,2)
status varchar(11)
last_studied_at? timestamptz
updated_at timestamptz
level_progress
PK id bigint
FK user_id bigint
hsk_level smallint
total_words integer
words_learned integer
characters_learned integer
percent_complete numeric(5,2)
updated_at timestamptz
earned_badges
PK,FK user_id bigint
PK,FK badge_id smallint
earned_at timestamptz
active_days
PK,FK user_id bigint
PK local_date date
recorded_at timestamptz
user_progress
PK,FK user_id bigint
current_streak integer
longest_streak integer
last_active_date? date
words_learned integer
characters_learned integer
total_sessions integer
updated_at timestamptz

Mockups

Landing page: Shows the value of HanziApp.





HanziApp

Funciones Niveles HSK

Inicia sesión

Todo lo que necesitas para aprender chino

Vocabulario por niveles HSK

Aprende de forma progresiva con contenido organizado por nivel y misiones de 20–30 palabras.

Escritura sin papel

Escribe caracteres directamente en pantalla y practica el orden correcto de los trazos.

Habilidad auditiva

Escucha, identifica y practica el vocabulario que estás aprendiendo.

Sin configuración

Empieza a estudiar en segundos. Sin mazos ni setups tediosos.

En español e inglés

Toda la interfaz en español. Pensado para hispanohablantes, no adaptado.

Crea tus propios mazos

Agrega tus propias palabras y practica con active recall. Tu aprendizaje, a tu manera.

HanziApp

Funciones Niveles HSK

Inicia sesión



Empieza hoy. Es gratis.

Tu camino para aprender chino empieza aquí.

Crear cuenta gratis →

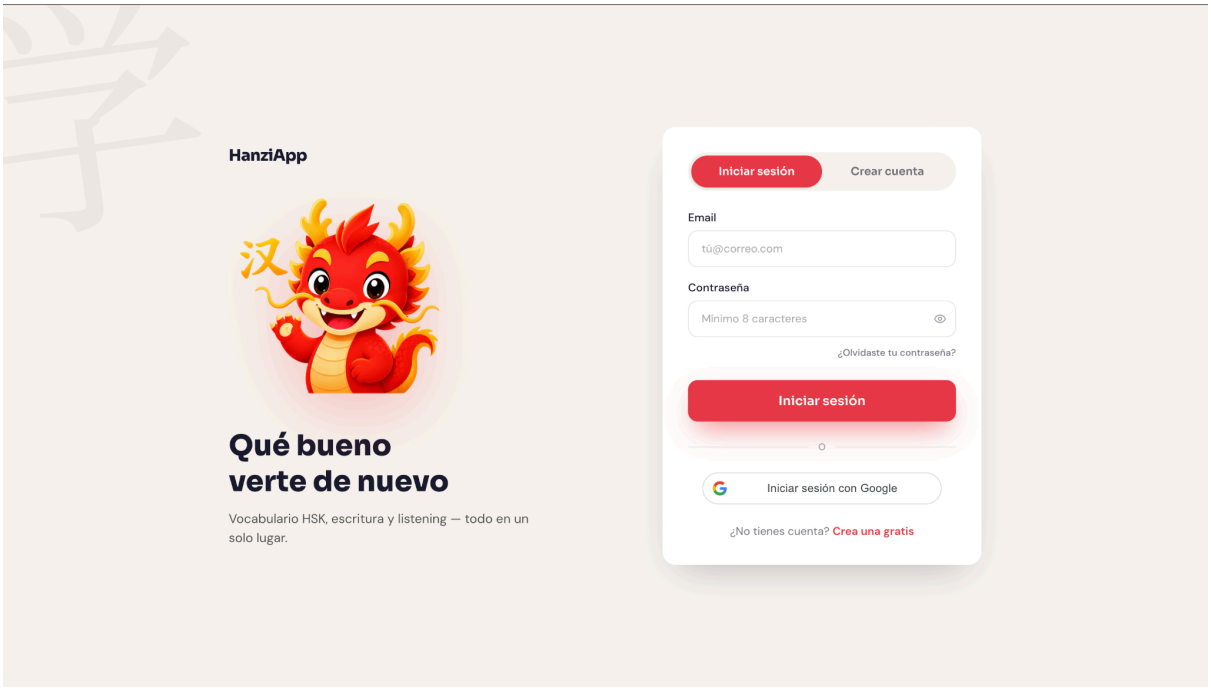
HanziApp

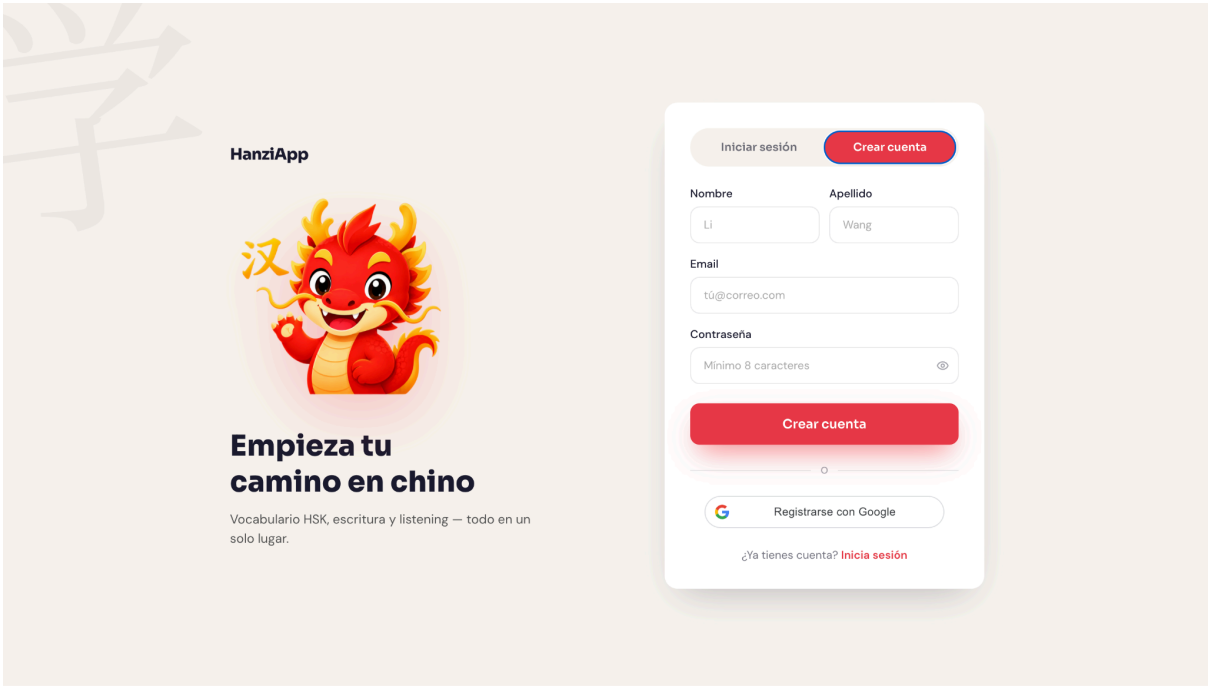
Funciones Niveles HSK Contacto

Hecho con ❤️ en Colombia 🇨🇴

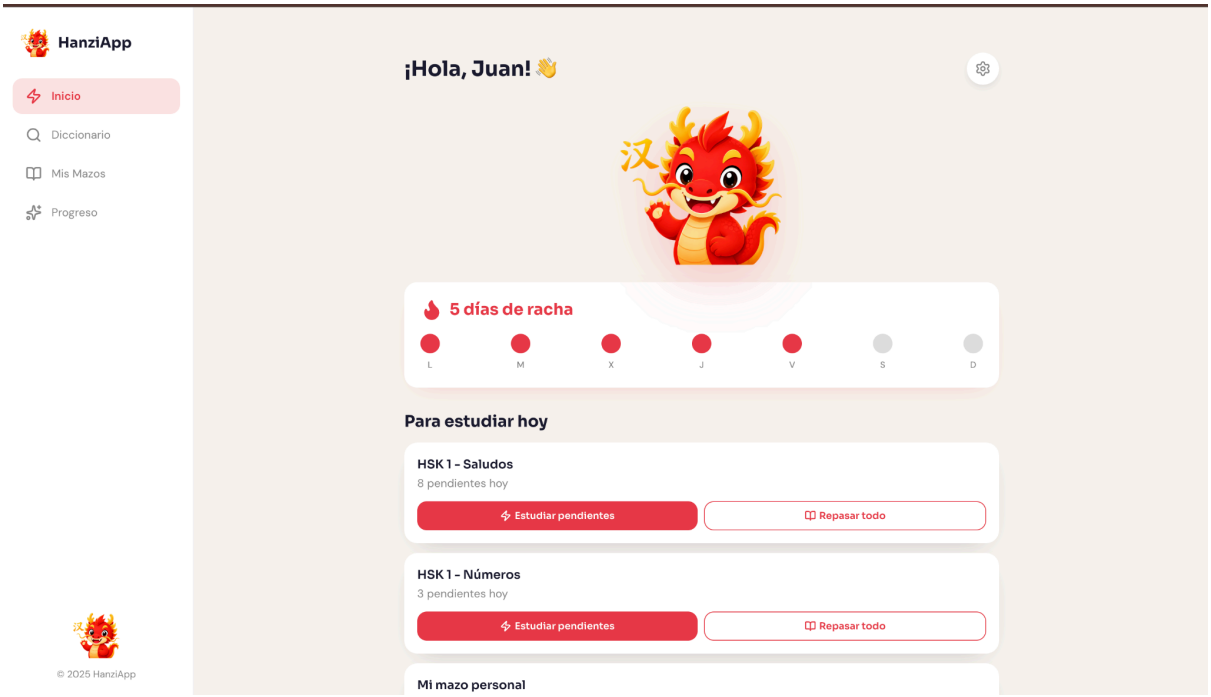
© 2025 HanziApp. Todos los derechos reservados.

Register and log in: Intuitive design for the user to log in or register.

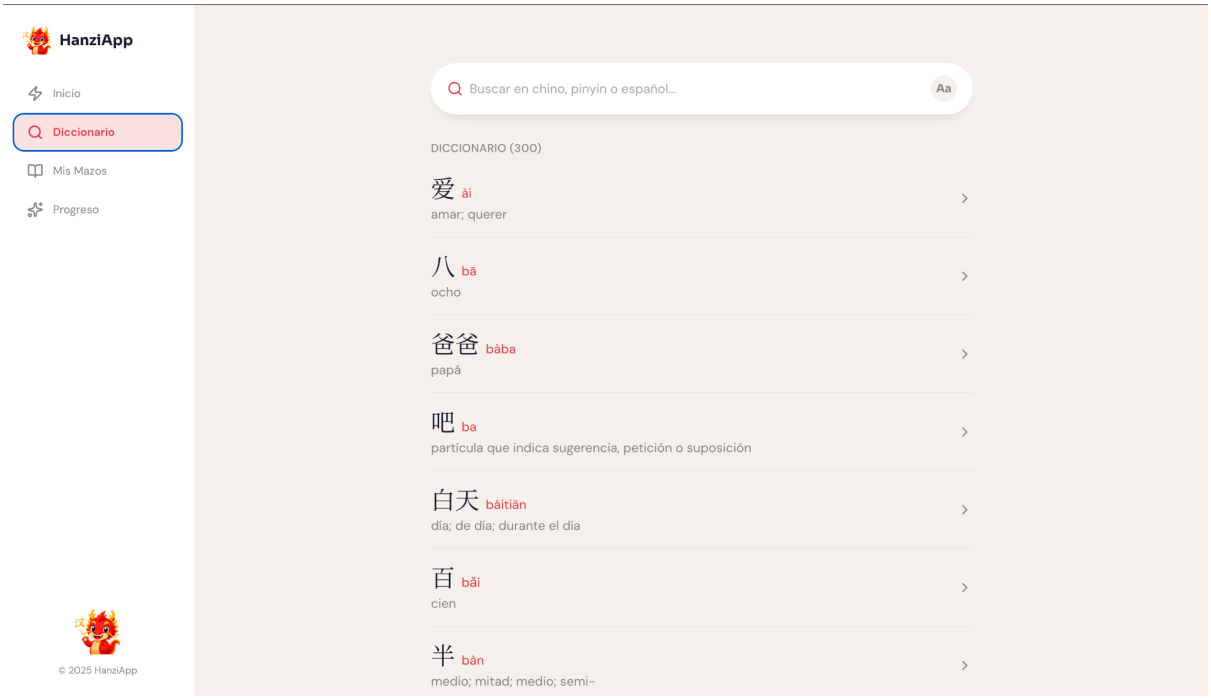




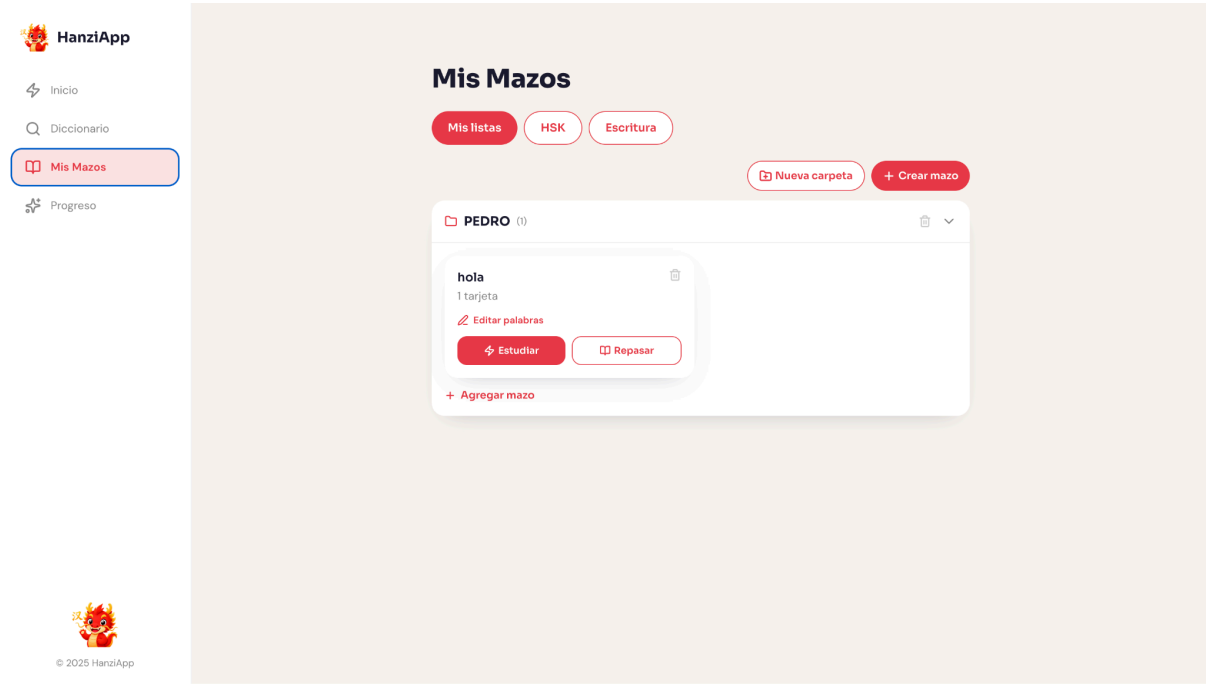
Main Page: Let the user look at his pending flashcards to study.

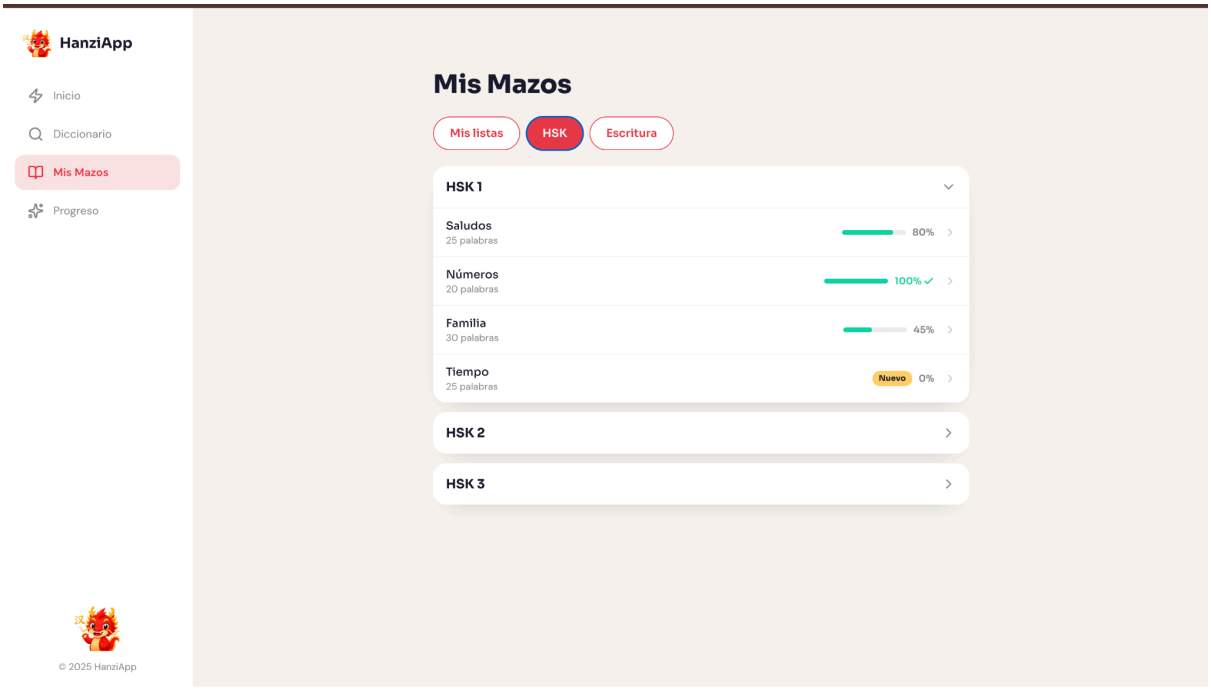


Dictionary: The user can look for words, meaning and add them to his decks.

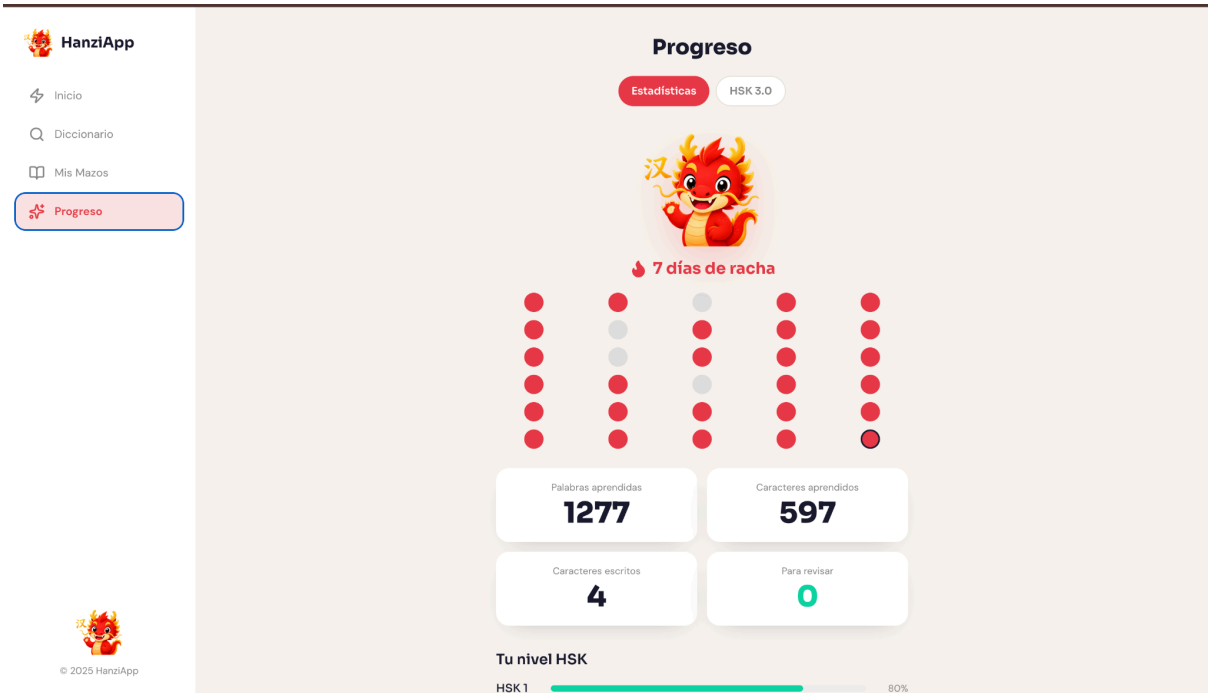


My decks: The user can control his lists and his decks.





Progress:. The user can look at his progress and his study streak.


HanziApp
Inicio
Diccionario
Mis Mazos
Progreso
© 2025 HanziApp
Progreso
Estadísticas
HSK 3.0
7 días de racha
Palabras aprendidas
1277
Caracteres aprendidos
597
Caracteres escritos
4
Para revisar
0
Tu nivel HSK
HSK 1
80%